KU LEUVEN

FACULTY OF
SOCIAL SCIENCES

Data-driven Organisation Design

A blockmodeling approach to organisational structure

Laura Nurski

0023565

Masters' thesis submitted to obtain the degree of

MASTER OF SOCIOLOGY

Supervisor: Prof. Dr. Alejandro Espinosa-Rada
Reporter: Bart Meuleman
Counselor: -
Academic year: 2025-2026
Word count: 22787

Acknowledgements

I am grateful to my supervisor, Prof. Dr. Alejandro Espinosa-Rada, for taking on this self-proposed topic and for doing so as a visiting professor, with thousands of miles and several time zones between us. Thanks to you I finally got to develop an idea I've carried with me for years.

I would also like to thank Seth Maenen for the multiple brainstorming sessions and feedback.

To my kids, Emmeline and Sophie: thank you for your patience, love, and support over the past three years. Combining this master's with a full-time job and mom duties was not always easy, but your kindness and cheerfulness have made this crazy endeavour bearable. I promise not to pursue any more masters until you're both 18 ☺.

Laura Nurski,
May 24th 2026,
Bierbeek

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Why structure matters and designing is difficult 1

1.2 The need for practical design tools 2

1.3 Contribution of this thesis 3

2 REPRESENTING ORGANISATIONS AS TASK NETWORKS 5

2.1 Network foundations for organisation design theory 5

2.2 Social networks vs task networks in organisations 7

2.3 Matrix-based representations of task-networks in the literature 8

3 APPLYING BLOCKMODELING TO ORGANISATION DESIGN 14

3.1 Nodes: tasks vs people 14

3.2 Links: types of task dependency 18

3.3 Criterion function: getting as close to ideal as possible 23

4 IMPLEMENTATION IN R AND SIMULATION STUDY 29

4.1 Implementation in R 29

4.2 Hypothetical production environment and hypotheses 31

4.3 Optimal partitions at different layer weights 33

5 RESULTS FROM EMPIRICAL DATA SET 40

5.1 Description of production setting and input data 40

5.2 Construction of the multilayer task-to-task network 41

5.3 Optimal partitions for cohesive product-machine groupings 43

6 DISCUSSION 51

6.1 Implications for organisation design practice 51

6.2 Limitations of the current analysis 52

6.3 Future research 53

7 CONCLUSION 55

8 REFERENCES 56

1 Introduction

Organisation design — and the organisational structures that give it form — is one of management's most powerful levers for translating strategic priorities into operational reality. **Organisation design** encompasses all the "*formal and informal organizational attributes, including structures, processes, and systems; their antecedents; and the influence they have on the organization, its constituent units, and its members*" (Joseph & Sengul, 2025, p. 249).

Organisational structure, as a defining element of that design, is "*the ways in which an organization divides its labour and integrates their efforts*" (Joseph & Gaba, 2020, p. 267; Mintzberg, 1979). It is the primary pathway through which firms align individual effort with overall strategy, by determining how activities are split into tasks; grouped into roles, teams and departments; and how the efforts of all these units are coordinated to achieve the goal of the organisation.

1.1 Why structure matters and designing is difficult

Decades of evidence show that this alignment between structure and strategy predicts **organisational performance**. Chandler's historical study showed that changes in corporate strategy (e.g. diversification into new markets) necessitate corresponding changes in organizational structure (Chandler, 1962). Classic contingency research demonstrated that structures that fit the firm's strategy and environment, perform better: mechanistic hierarchies suit stable settings, while organic, decentralised arrangements support innovation and adaptation under uncertainty (Burns & Stalker, 1961; Lawrence & Lorsch, 1967). Recent economic models and econometric evidence confirm these ideas: decentralised firms outperform centralised ones in turbulent bad times (Aghion et al., 2021), because local decision authority enables faster adjustment (Dessein & Santos, 2006).

Structure also affects **worker wellbeing** by channelling workloads, shaping autonomy and role clarity, and affecting organisational participation. Notably, a recent extensive review of the antecedents of job design, finds that *organisational*-level influences (such as structure) have the most clear and consistent evidence for shaping the design of individual jobs, over the *supra*-organisational (e.g. national and occupational) and *infra*-organisational (e.g. work group and individual) influences, "reflecting the proximity of these factors to local work design" (Parker et al., 2017). Wellbeing experts agree that higher job satisfaction in flatter, less hierarchical organisations is due to greater job control and trust, while excess layering can reduce engagement and increase stress (Powdthavee & Frijters, 2017). Because healthier, more engaged employees are also more productive and innovative, structure is a core lever for building both high-performing firms and resilient workforces (Barrick et al., 2015; Bryson et al., 2017; Harter et al., 2010).

Given the importance of structure for organisational performance, Greenwood and Miller (2010, p. 78) argue that understanding organisation design "should be a core, perhaps even the core, concern of organisational theory." They note that design has become harder to study as **organisations have grown more complex**, operating across multiple geographies, professions and stakeholder environments. Yet the topic has not received commensurate attention. The authors attribute this to the "intimidating complexity" of organisation design: understanding modern structures requires detailed, close-range empirical work rather than pre-assembled databases, and such work demands access and resources that are not easily secured (Greenwood & Miller, 2010, p. 80).

Recent scholarship argues that organisations should be designed from the ground up, starting from the activities and tasks that constitute organisational processes and the interdependencies among them (Puranam, 2018; Raveendran et al., 2020). From this **microstructural perspective**, structure is not

merely the arrangement of departments or reporting lines but a pattern of interdependent, relational interactions that must be coordinated. This view has deep roots: March and Simon (1958) established the centrality of interdependence in organisations, and Simon (1962, p. 468) described them as hierarchically decomposable systems of interacting parts — from departments to activities to tasks, all the way down to what can be considered the "elementary particles" of organisational work.

Operationalising this approach is demanding. Most organisations involve hundreds of distinct activities, decomposable into thousands of tasks. The relationships among them are often undocumented, or knowledge about them is dispersed across the organisation. Moreover, the same pair of activities can be connected in multiple ways simultaneously: one may depend on another's output as one's own input while also sending information back, and a third may need to oversee both. Turning this multidimensional web of task interdependencies into a coherent organisational structure is an analytically difficult problem. As a result, the microstructural approach is mostly confined to theory-building, laboratory experiments, or agent-based simulation, and rarely applied to empirical datasets of real organisations (Joseph & Gaba, 2020, p. 292).

Beyond data demands, a further conceptual challenge is that there is **no single grouping principle** for determining which tasks or activities should be grouped in a team or department. Different theoretical traditions emphasise different rationales — specialisation, resource optimisation and economies of scale, coordination of and-to-end workflow interdependence, and separation of conflicting goals — and in real organisations these rationales coexist and often pull against each other. Grouping similar or parallel activities may conflict with grouping interdependent or sequential ones; separating conflicting roles may conflict with both. Design therefore involves **navigating tensions** rather than applying a single rule. The resulting trade-offs — for example between searching for better designs and maintaining stability — are well documented (Ethiraj & Levinthal, 2004; Rivkin & Siggelkow, 2003), but rarely translated into data-driven tools that help practitioners make concrete structural decisions.

1.2 The need for practical design tools

Organisation design scholarship provides rich theoretical insight but has not translated it into methodological tools that practitioners can use.

Traditional data sources used in academic research on organisational structure are too coarse for microstructural analysis. Management surveys, which underlie most of the quantitative empirical literature, only capture macro-structural constructs such as centralisation, formalisation or divisionalisation — which are too broad to inform fine-grained design decisions (Albert et al., 2025). Other sources in the empirical literature are even more indirect, using patent assignments or executive titles as proxies for structural characteristics (Albert et al., 2025). None of these offer insight into the relational architecture of tasks and interdependencies that underlies organisational functioning.

Organisational network analysis (ONA), as a subfield of social network analysis (SNA) offers a more targeted alternative. Rather than capturing macro firm-level attributes from management surveys, ONA collects relational data through employee surveys that directly map communication, advice-seeking and collaboration patterns among employees (Borgatti & Foster, 2003; Cross et al., 2004). These data – capturing who works with whom, on what, and how often – provide precisely the kind of relational architecture that blockmodeling requires. Yet these data still describe the *social* network that has formed around an existing organisational structure — not the underlying *task* network that should inform the design of a new structure (see sections 2.2 and 3.1 for an in-depth discussion of these two types and how they are linked).

Data limitations are reflected in the prevalence of ONA applications across **analytical levels**. A review of the ONA literature reveals a striking imbalance across levels of analysis: more than half of the reviewed material focuses on *interorganisational* networks (i.e. between organisations), and over a third on *interpersonal* networks inside organisations (i.e. between people), whereas *interunit* networks—where organisational units or groups are treated as actors—receive only limited attention (Brass et al., 2004). Yet it is exactly this interunit level that corresponds most directly to the analysis of organisational structure, and it is currently underexplored.

Advances in big data collection and processing, including machine learning, natural language processing and topic modelling of communication between organisational members, further open up possibilities for modelling complex organisations and capturing the trade-offs of different structural choices more directly (Joseph & Gaba, 2020, p. 293).

Yet practitioners still **lack methods** that translate detailed relational information into design recommendations. Puranam (2025, p. 83) calls the current state of "data-free" organisation design "the norm". A practitioner-oriented book on *Data-Driven Organization Design* illustrates the gap well: it maps activities to roles across different grouping bases and visualises the resulting role interactions in a relational matrix (Morrison, 2021, p. 171), coming close to the microstructural analysis this thesis pursues. Yet despite the relational framing and the title of the book, no mathematical method is offered for selecting the best grouping.

Blockmodeling fills this gap. Rooted in social network analysis, it can group nodes — whether individuals or activities — according to their patterns of relationships and characteristics, producing interpretable clusters and structural representations (Doreian et al., 2005). These clusters can reflect functional roles or structural positions based on different types of equivalence sought among the nodes. Blockmodeling also allows researchers to specify functions that determine the principles by which nodes should be grouped, making it possible to integrate multiple theoretical rationales in a single analytical framework. With the growing availability of relational data within organisations, it offers a mathematically transparent foundation for data-driven organisation design.

Nelson (1986) was the first to propose applying blockmodeling to the analysis of organisation structure, arguing that it studies "*directly the multiple kinds of interactions which constitute organisation structure rather than relying on abstractions of global properties (centralisation, formalisation, span of control, etc)*" (Nelson, 1986, p. 75). Unfortunately, the idea attracted **little follow-up**: Google Scholar lists only 24 citations to his 1986 paper. Overall, the application of blockmodeling to organisation design has been rare. A Web of Science topic search on 'blockmodel' and 'organisation structure' found no results[1], while the same search on Scopus only found 5 papers, including the Nelson 1986 paper[2].

1.3 Contribution of this thesis

Nelson's proposed use of blockmodeling was **descriptive** in orientation: a blockmodel of observed communication data would *reveal* the structure of the organisation – which in turn could be used to empirically test theoretical relationships between structure, strategy and environment. Applications of SNA and ONA in general have largely maintained this stance: uncovering the organisational structure from social network data. For organisation design purposes, however, one does not wish to *reveal* the structure, but to *design* one. From this perspective, observed interaction data is the wrong starting point: social or informal networks reflect the formal structure already in place, not the structure that the work actually requires (see section 2.2).

[1] Query on WoS: TS = (blockmodel*) AND TS = ("org* structure" OR "org* design"), conducted on 15/12/2025.
[2] Query on Scopus: (TITLE-ABS-KEY (blockmodel*) AND TITLE-ABS-KEY ("org* structure" OR "org* design")), same date.

This thesis therefore takes a different direction and applies blockmodeling **prescriptively**. Rather than uncovering social structure from observed interactions, it searches for optimal partitions of the underlying task network in order to design an organisation structure. The model considers which tasks are interdependent in a larger sequential workflow, share knowledge requirements or other resources, or potentially conflict in their goals. Starting from the task network is common in other disciplines (like engineering and management science), but these fields have little connection to SNA, ONA or blockmodeling. The closest precedent is Ethiraj and Levinthal (2004), who similarly start from the task interaction structure – but use it to simulate how boundedly rational actors *search for* a good design, not to identify the rationally optimal one directly, and they never mention blockmodeling.

This thesis incorporates organisation design theory directly into the blockmodeling criterion (i.e. the objective function to optimise). The **central contribution** is a generalised blockmodeling framework for organisation design that (1) integrates three rationales for grouping or separating activities — functional specialisation, coordination of workflow interdependence, and separation of conflicting goals — each represented as a distinct link layer in a multiplex task network, and (2) assigns weights to those rationales according to the organisation's strategic priorities — operationalising the principle that structure should follow strategy. For practitioners, the method offers a structured way to explore structural options: organisations can examine their task system under different strategic priorities and compare alternative groupings before implementing changes.

This thesis is structured as follows. **Chapter 2** establishes the relational foundations of the framework: it reviews relational approaches to organisation structure, distinguishes social from task networks, and examines existing matrix-based representations of task networks in the literature. **Chapter 3** reviews the blockmodeling framework from an organisation design perspective: it defines tasks as nodes, specifies three types of task dependency as link layers, and constructs the criterion function from a strategy-structure perspective. **Chapter 4** operationalises the framework in R and applies it to simulated data in a hypothetical production setting. **Chapter 5** applies the framework to real data from a Belgian FMCG factory looking to reorganise from nine functional departments to three autonomous end-to-end product teams. **Chapter 6** discusses limitations, implications and directions for future research. **Chapter 7** concludes.

2 Representing organisations as task networks

Getting to the micro-foundations of organisational structure requires studying the pattern of interactions between the tasks and activities that make up the work. A relational framework is well suited to this: network analysis treats structure as a pattern of ties rather than a collection of attributes, and can represent tasks, their links or dependencies, and the people who carry them out at multiple levels of analysis simultaneously.

This chapter builds the representational foundations for doing so. Section 2.1 establishes why network concepts map coherently onto organisational design theory. Section 2.2 argues why the task network — rather than the social interaction network that forms on top of it — is the appropriate object of analysis when the goal is to *design* structure rather than *describe* it. Section 2.3 documents how several disciplines have independently converged on matrix-based representations of task networks, arriving at the same formal logic through entirely separate intellectual traditions.

2.1 Network foundations for organisation design theory

The network as a relational perspective offers a coherent representational foundation for analysing organisational structure.

First, in network analysis, **relations** constitute the primary unit of analysis, and outcomes are explained by actors' positions within a relational structure rather than by their isolated characteristics (Marin & Wellman, 2014). This relational stance mirrors a central insight in organisation theory: once labour is divided across tasks, roles, or units, organisational performance depends on how the resulting interdependencies between organisational units are managed and coordinated (March & Simon, 1958; Thompson, 1967).

A second point of alignment concerns the **choice of nodes**. In network analysis, nodes generally represent different kinds of actors depending on the level of analysis, ranging from individuals to organisational units or entire organisations (Brass et al., 2004), but the framework also allows nodes to be artifacts, activities or events. Networks can even be drawn between two or more sets of different nodes. For the study of organisational structure, nodes likewise need not be limited to people or groups. They can also represent tasks or activities, either in single-mode networks (tasks linked to tasks) or in multi-mode networks that connect tasks to roles, teams or organisational units.

A thirds correspondence concerns the **macro-micro link** across such levels of analysis. In network analysis, macro-level properties such as segregation, clustering or brokerage emerge from the combination of micro-level relations and positions (Duxbury, 2024). The overall network takes shape over time through different micro mechanisms — structural processes such as reciprocity, transitivity, homophily, and preferential attachment that regularly generates ties (Stadtfeld & Amati, 2021). The microstructural approach in organisation theory makes an analogous move when it treats macro-structural characteristics — such as span of control, centralisation of decision-making, or hierarchical depth — as aggregates of underlying coordination ties. Network representations therefore provide a natural way to capture the micro-structural foundations of organisational structure and aligns with Brass et al.'s (2004) call to study how micro interactions and macro organisational forms are linked.

A fourth alignment lies in the treatment of **multiplicity of ties**. Network analysis allows multiple types of relations to coexist between the same nodes, giving rise to multiplex networks in which actors may be simultaneously connected through communication, advice, trust, conflict, or other relations. Organisation theory similarly recognises that tasks can be connected through different types of

interdependence — such as task, goal, or knowledge interdependence (Raveendran et al., 2020). Task interdependence can even be 'negative' when related tasks have conflicting functional goals (Worren & Pope, 2024). Representing these connections as distinct relational layers avoids collapsing heterogeneous coordination and conflict problems into a single structural dimension.

Fifthly, network analysis, and specifically blockmodeling, offers a precise language for distinguishing **positions** in the network from the **roles** they perform (Ferligoj et al., 2014). Structural equivalence identifies actors with identical ties to the same actors (thus taking up a similar position), while regular equivalence identifies actors with similar ties to similar actors (thus taking up a similar role). This distinction maps directly onto organisation design, as some people might take up different positions (in different teams), while performing the same role (i.e. administrative support). Popular organisation designs also reflect position-based grouping (like a divisional firm – grouping activities that are connected through a shared product, market or customer) and role-based grouping (like a functional organisation – grouping activities that perform a similar function, such as HR, IT or finance).

Finally, the network perspective aligns closely with how organisation design conceptualises **complex system architecture**. Network analysis treats communities and hierarchies as emergent properties of relational patterns, with modularity capturing dense within-group and sparse between-group connections (Freeman, 2014) [3]. Organisation theory similarly views organisations as nearly decomposable systems (Simon, 1962), in which teams or units interact more intensively internally than externally. From this perspective, the boundaries of organisational units can be reinterpreted as the outcome of partitioning a network of interdependent activities into modules that balance internal and external coordination demands.

Table 1 summarises the above-described alignment between network analysis and organisation structure. Taken together, these correspondences show that network analysis is not merely a methodological add-on to organisation design theory, but a representational framework that captures its core foundational concepts.

Table 1 Network analysis vs organisation design: Basic principles and measurements

	Network analysis	**Organisation theory**
Relational perspectives	Outcomes are explained via patterns of ties and positions in a relational structure. Relations are the primary unit of analysis.	Organisational performance depends on the coordination of interdependent activities among individuals, teams, or units arising from the division of labour.
Types of nodes	Nodes that make up the network can be individuals, groups, institutions, artifacts, activities or events.	Nodes can be tasks (and their aggregations like activities and processes) or people (and their aggregations, like teams or departments).
Aggregation across levels	Global properties (network level measures), such as centralisation or assortativity, arise from the aggregation of local properties	Macro-structural characteristics, such as centralisation of decision-making, hierarchy or span of control, arise from the

[3] Throughout this thesis 'modularity' refers to the architectural property of near-decomposability following Simon (1962) and the DSM tradition, and is distinct from Newman and Girvan's (2004) modularity Q, a community detection metric that quantifies this property against a random-graph null model.

	(node or dyad measures), such as closeness or betweenness.	aggregation of micro-structural ties in the organisational network.
Multiplicity of ties	The same network can contain multiple types of ties between the same actors (liking/disliking, communication, advice, trust, …), leading to multiplex networks.	The same set of tasks can be connected in multiple ways through different types of interdependence, different types of similarity and different types of goal conflicts.
Position vs role	Distinguishes positions via *structural equivalence* (same ties to the same others) and roles via *regular equivalence* (similar ties to equivalent others).	Considers position-based partitions based (e.g. end-to-end product teams or business units) and role-like grouping based on similarity (e.g. functional roles).
System architecture	Uses concepts like communities and hierarchy as emergent network properties. Modularity captures dense-within / sparse-between partitions.	Views organisations as nearly decomposable / modular systems where subsystems interact more strongly internally than with other subsystems.

2.2 Social networks vs task networks in organisations

Organisational network analysis (ONA) is the application of social network analysis (SNA) to organisations. It has grown from near zero publications in 1970 to around 450 per year by the turn of the century, driven by a broader shift away from individualist explanations towards more relational ones (Borgatti & Foster, 2003). That growth has developed primarily around the study of ***social* interaction networks** — the ties of communication, advice, collaboration, and trust that form among individuals, teams, and organisations. Whether explaining why some actors outperform others, why practices spread through a system, or how firms converge on similar structures, the networks under analysis are often built from observed interaction data: records of who interacts with whom or at which events (Borgatti & Foster, 2003; Brass et al., 2004).

Using social network data for the design of organisational structures creates a practical problem, however. **Formal structure** is itself one of the strongest antecedents of informal interaction: physical co-location, reporting relationships, and team or project assignments act as *foci* (Feld, 1981) that concentrate contact opportunities and shape who interacts with whom (Brass et al., 2004). The social network in an organisation therefore partly reflects the structure already in place. Furthermore, these organisational structures may not even be the most efficient ones, as organisations often adopt structures to conform to field-level expectations (DiMaggio & Powell, 1983) — regulatory pressures, professional norms, or imitation of perceived best practice. Observed interaction patterns consequently reveal what formal structure is already there; they cannot prescribe which organisational structure would fit the work activities best.

Contingency theory offers a different lens for designing optimal organisational structures. It has extensively documented how **the right organisation structure is contingent on the structure of the task environment.** Designers should therefore match or fit the organisational structure to the coordination and information demands generated by the characteristics of the task network itself (Galbraith, 1974; Thompson, 1967). The relevant network for *design* is therefore the task network —

which tasks are interdependent in larger workflows, require similar knowledge or use shared resources, or potentially conflict — rather than the social network that has formed around whatever structure currently exists.

The point where task and social networks intersect can be a place of collaboration and **consensus**, or it could be a point where interests and goals **conflict**. In the mid-20th century works that founded organization design, scholars traditionally viewed goal and knowledge interdependence as "essentially congruent with task interdependence", while contemporary scholarship treats these types of interdependence as decoupled or conflicted (Raveendran et al., 2020). The importance of building teams (or other organisational units) around task structures rather than around interpersonal preferences is illustrated in a meta-analysis of almost 200 studies over 10.000 individuals: task cohesion (the extent to which the shared task allows the group to attain important goals) is a stronger prediction of team performance than social cohesion (liking or attachment to the members of the group) (Grossman et al., 2022).

This thesis thus applies blockmodeling to the task network to identify the partition that best groups together tasks with high coordination needs between them, so that the units that are assigned those bundles of tasks must work together internally (not externally) to achieve their goals. Blockmodeling is used here not in a **descriptive** way — to uncover existing communities among organisational participants — but in a **prescriptive** way: to partition the task network as a basis for building the organisational structure. The goal is not to recover how the organisation is currently built, but to determine what its optimal structure should be.

The closest precedent in the literature is Ethiraj and Levinthal (2004), who also use the task structure as input for the design of the organisational structure, but focus on how boundedly rational actors search for the optimal grouping of tasks, (see section 2.3.4), while this thesis assumes a fully rational organisation designer. Another close reference is Collins et al. (2010) who track how a product development task network changed before and after the reorganisation of a small engineering firm — but their analysis remains diagnostic, aimed at understanding what the task structure became, not at prescribing what it should be.

2.3 Matrix-based representations of task-networks in the literature

Because network analysis provides a natural representation of the organisational task network, several disciplines have converged, largely independently, on **matrix-based representations** as a formal way to operationalise these relational patterns. Matrices offer a compact and analytically tractable means of representing complex systems of dependence, allowing relational structures to be analysed, compared, and manipulated systematically. As a result, matrix representations of organisational task structures appear across socio-technical systems analysis, product engineering, factory layouts and, more recently again, organisation science.

2.3.1 Socio-technical systems analysis: designing work processes

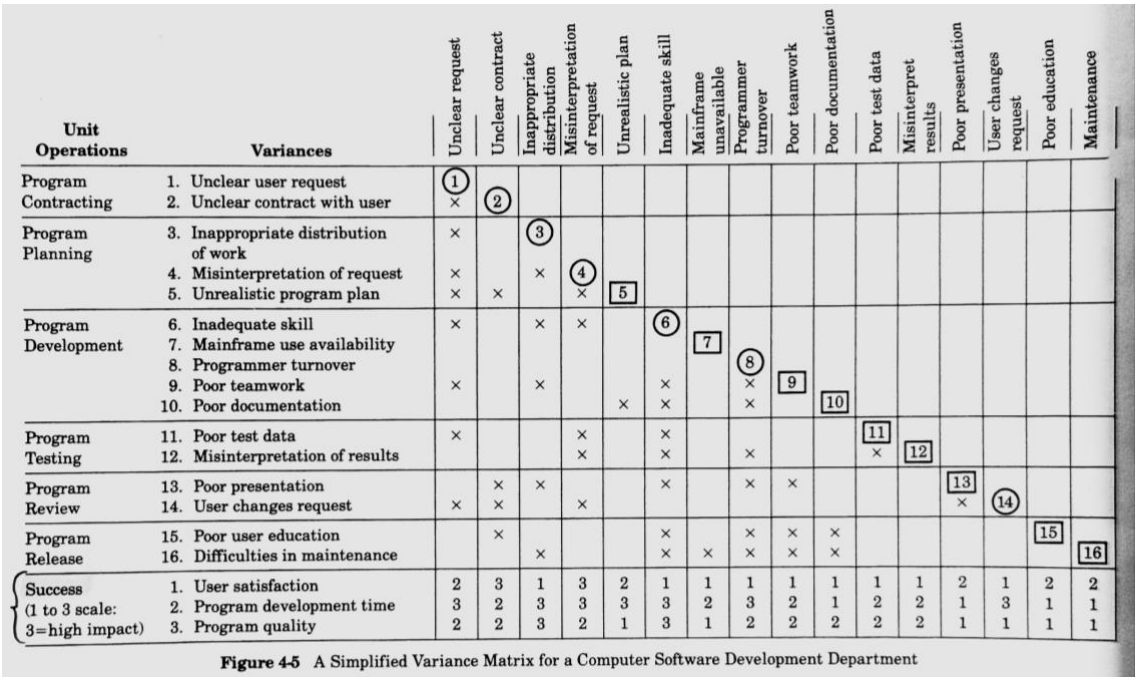
Early “hard systems” thinkers were among the first to attempt a formal, quantitative representation of organisational task structures. Within **socio-technical systems** (STS) theory, organisation design was approached as a problem of system regulation and control, grounded in Ashby’s law of requisite variety (Ashby, 1958). The central claim was that organisations could function effectively only if their internal control structures possessed sufficient variety to match the variability generated by tasks and the environment.

The **variance matrix** (see Figure 1) emerged as a concrete operationalisation of this logic. It maps organisational units and their activities against specific sources of operational variance, indicating where deviations originate and how they propagate across interdependent components of the organisation (Pasmore, 1988, p. 75). The variance matrix highlights vulnerabilities in organisational design: the size of the variance at each node (the task) – and its propagation across the next nodes signals mismatches between task complexity and local capacity for detection, interpretation, and response.

Despite its conceptual sophistication, this matrix-based approach never became mainstream in STS studies. Its practical application was constrained by limited data availability and computational limitations at the time. Large-scale attempts to implement cybernetic control principles in real organisations – notably Stafford Beer's Cybersyn Project in Chile (Medina, 2011) – exposed the political, informational, and operational challenges of translating such highly formalised control architectures into practice (Lehto & Paniagua, 2025).

Over time, STS developed into distinct national traditions that diverged precisely on this question of formalisation. The North American strand shifted its emphasis toward participative design and quality of working life (Cherns, 1976; Mumford, 2003), and has more recently intersected with constructivist currents in science and technology studies (Bijker, 1999) that resist formal representations of organisations altogether. The Lowlands tradition — developed in the Netherlands and Flanders (de Sitter et al., 1994; Van Hootegem, 2019) — retained a more objectivist, design-engineering orientation in which organisational structure is treated as an object amenable to formal analysis and optimisation[4].

As a result, matrix-based STS approaches remained influential conceptually, but marginal in empirical organisational research.



Unit Operations	Variances	Unclear request	Unclear contract	Inappropriate distribution	Misinterpretation of request	Unrealistic plan	Inadequate skill	Mainframe unavailable	Programmer turnover	Poor teamwork	Poor documentation	Poor test data	Misinterpret results	Poor presentation	User changes request	Poor education	Maintenance
Program Contracting	1. Unclear user request	(1)															
	2. Unclear contract with user	×	(2)														
Program Planning	3. Inappropriate distribution of work	×		(3)													
	4. Misinterpretation of request	×		×	(4)												
	5. Unrealistic program plan	×	×		×	[5]											
Program Development	6. Inadequate skill	×		×	×		(6)										
	7. Mainframe use availability							[7]									
	8. Programmer turnover								(8)								
	9. Poor teamwork	×		×			×		×	[9]							
	10. Poor documentation					×	×		×		[10]						
Program Testing	11. Poor test data	×			×		×					[11]					
	12. Misinterpretation of results				×		×		×			×	[12]				
Program Review	13. Poor presentation		×	×			×		×	×				[13]			
	14. User changes request	×	×		×									×	(14)		
Program Release	15. Poor user education		×				×		×	×	×					[15]	
	16. Difficulties in maintenance			×			×	×	×	×	×						[16]
Success (1 to 3 scale: 3=high impact)	1. User satisfaction	2	3	1	3	2	1	1	1	1	1	1	1	2	1	2	2
	2. Program development time	3	2	3	3	3	3	2	3	2	1	2	2	1	3	1	1
	3. Program quality	2	2	3	2	1	3	1	2	2	2	2	2	1	1	1	1

Figure 4-5 A Simplified Variance Matrix for a Computer Software Development Department

Figure 1 Variance matrix in socio-technical systems analysis (Pasmore, 1988, p. 75)

[4] William Pasmore is a notable bridge figure: his empirical survey of North American STS practice (Pasmore et al., 1982) is firmly in the participative tradition, yet his own methodological work—including the variance matrix (Pasmore, 1988)—adopts the objectivist, design-analytical stance more characteristic of the Lowlands school.

2.3.2 Engineering complex systems: designing products and components

The **Design Structure Matrix** (or Dependency and Structure Modelling, DSM) approach originated in the fields of engineering and systems design, with early foundational work by Donald Steward in the 1970s and 1980s (Steward, 1981). Steward developed the DSM as a tool to represent and analyse the dependencies between components in complex systems, such as product architectures or project tasks. His work was further popularized and expanded by researchers at MIT, notably Steven Eppinger and Tyson Browning (Browning, 2001; Eppinger & Browning, 2012). Initially applied to product architectures, DSM methods were designed to manage tightly coupled technical systems (like airplane engines) by identifying interaction patterns that could be reorganised into more modular structures.

The DSM gained traction because it offered a pragmatic way to model complex system interdependencies using relatively sparse qualitative or semi-quantitative data (DSM Conference, 2019), and because it could be implemented with modest computational resources. Its success in engineering contexts – also notably in modular product and computer system design (Baldwin & Clark, 2000) – stood in sharp contrast to the limited diffusion of earlier cybernetic approaches in organisation theory. While rooted in engineering, DSM methods were gradually extended to organisational contexts, particularly in product development settings where task, process, and team interdependencies are closely intertwined. As illustrated in Figure 2, DSMs can be used to represent product, organisational, and process architecture separately, or to integrate all three networks into a single linked network.

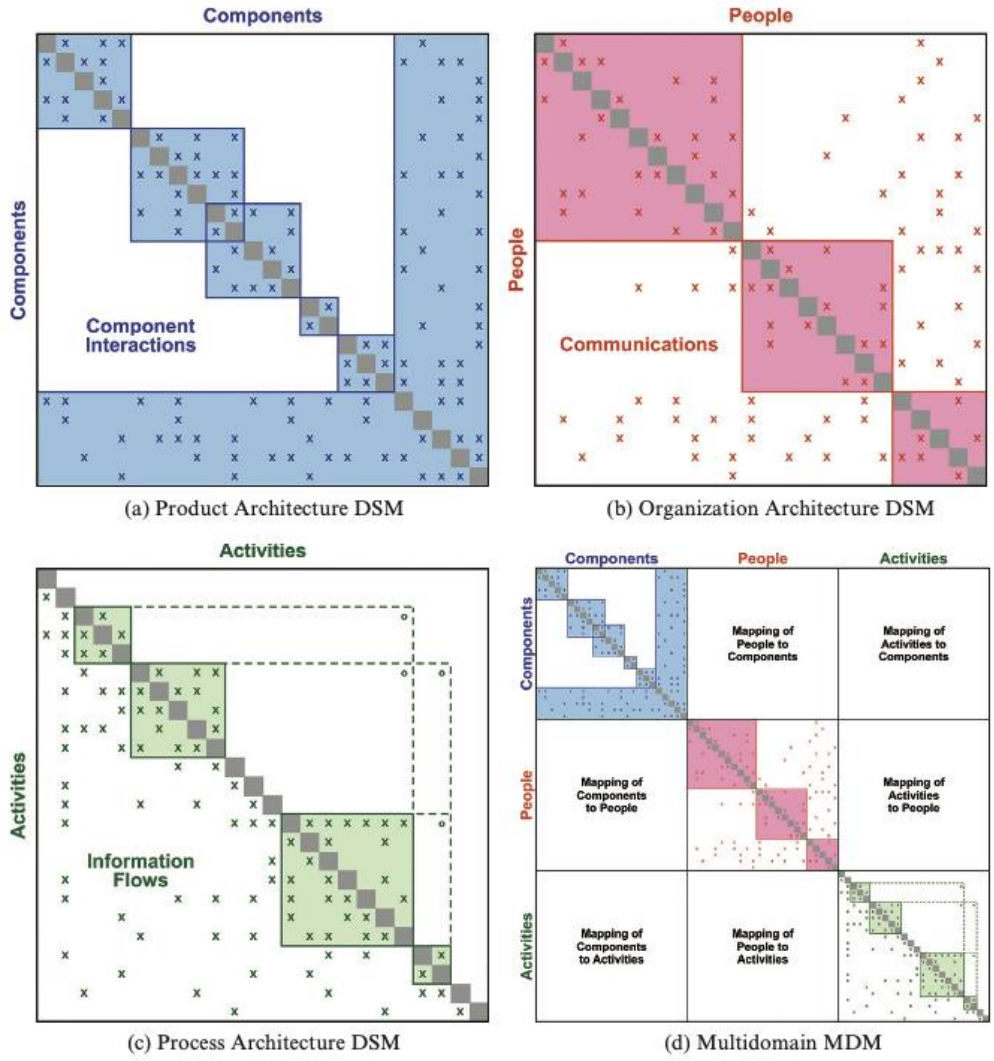


Figure 2 Product, organisation and process DSM from Eppinger & Browning (2012, figure 1.1)

While the DSM community recognises the diverse roots of the discipline (Browning, 2001, p. 293), the field retains a strong disciplinary imprint. The focus is almost exclusively on interdependence as the dominant grouping rationale, aiming to cluster tightly coupled elements and minimise cross-cluster interactions. Other organisation theoretic concerns – such as specialisation of expertise, goal conflict, or trade-offs between them – are largely outside the scope of current DSM practices.

2.3.3 Production flow analysis: designing factory layouts

A parallel development in industrial engineering arrived at the same matrix partitioning logic but applied to a different aspect of production – factory layouts. **Production flow analysis (PFA)** was developed by Burbidge (1991) as a systematic method for reorganising factory layouts from functional arrangements (where machines of the same type are grouped together) to cellular layouts (where machines handling the same part families are co-located) (Hameri, 2011). The driving insight was that apparent complexity in a factory often dissolves once the right groupings are found. What looks like an entangled network of flows across the shop floor can – with the right partitioning of machines – resolve into a small number of near-autonomous cells.

The core representation is the machine-part matrix, in which rows correspond to machines and columns to parts, with a mark in cell (i, j) indicating that machine i processes part j. An unordered matrix reveals little about the natural structure of the operation, representing the 'spaghetti'-like movements across the factory floor. The analytical task is to find a permutation of rows and columns that places all marks as close to the diagonal as possible, so that each cluster on the diagonal defines a production cell of machines and the parts they produce. Off-diagonal marks are exceptions representing inter-cell movements and minimising them is the practical objective of layout redesign.

Figure 3 illustrates the method with a back-office case in which workers play the role of machines and insurance claim types play the role of parts (Hameri, 2011). The left matrix shows eight workers and twelve insurance claim types in arbitrary order; the right matrix shows the same data after reordering rows and columns, with two blocks visible on the diagonal. These two blocks represent two teams of workers, together with the types of claims they handle. Two exceptions remain and are circled in the reordered matrix, representing two types of claims that are handled by both teams. The count and location of such exceptions determine whether reorganisation is feasible: few isolated exceptions yield clean, largely autonomous cells, while a high exception rate signals interdependence that a cellular layout cannot resolve. If clustering results in too many exceptions, management could change the product design, re-plan machining procedures, or outsource the exceptional parts.

Claim / Worker	1	2	3	4	5	6	7	8	9	10	11	12
A		x	x	x			x	x			x	x
B	x			x					x	x		
C			x				x	x			x	
D	x			x	x	x						
E						x			x			
F		x	x	x			x		x			x
G	x					x			x	x		
H		x	x	x			x				x	

Claim / Worker	1	9	10	5	6	7	2	11	3	8	4	12
G	x	x	x		x							
D	x			x	x							
B	x	x	x								x	
E		x			x							
A						x	x	x	x	x	x	x
F		x				x	x		x		x	x
C						x		x	x	x	x	
H						x	x	x	x		x	

Figure 3 Clustering in Production Flow Analysis (Hameri, 2011) – unordered (left) and ordered (right)

This clustering of parts or products was formalised in PFA as an optimisation algorithm modelling the machine-part matrix as a bipartite graph and using spanning trees and strongly connected components to identify natural blocks (Daita et al., 1999). The resulting 'Block Diagonal Form' — a partition of rows and columns into sets such that the off-diagonal blocks are as sparse as possible — is in structural

terms, **identical to two-mode blockmodeling**: both seek a partition of relations into internally dense, externally sparse blocks. Yet neither the PFA literature nor the blockmodeling literature cites the other – the two bodies of work reached the same formal solution through separate intellectual lineages.

2.3.4 Organisation science: designing organisational structures

From the early 2000's onwards, the management science community embraced the DSM when two Harvard professors of business administration published a book called "Design Rules, Volume 1. The Power of Modularity" (Baldwin & Clark, 2000). Exploring the process of design and its profound impact on the structure of industries, the authors explicitly rely on the work of Steve Eppinger and Donald Steward, founding fathers of the DSM. While the word 'block' appears on 45 pages of the book and even block structures are discussed, the blockmodeling literature is not referenced at all.

Matrix-based representations of task networks, like the DSM, have been particularly influential as tools for theory building and computational modelling. Ethiraj and Levinthal (2004), for example, use a DSM to model loosely versus tightly coupled task systems, combined with hierarchical and non-hierarchical coordination structures, to study adaptive dynamics in organisational design, see Figure 4. They simulate how boundedly rational actors search for a good grouping of these tasks through local operators — splitting departments, combining them, and transferring individual functions — applied to a fitness landscape they cannot directly observe. While the authors never mention blockmodeling in this paper, their algorithm shows remarkable similarities with community detection algorithms, namely splitting clusters, combining clusters and moving nodes from one cluster to another.

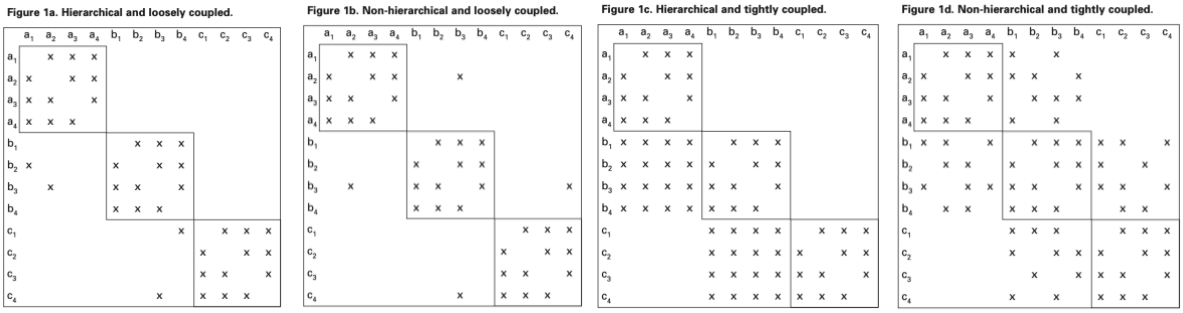


Figure 4 Matrix representations of organisation structure for computational modelling (Ethiraj & Levinthal, 2004, pp. 13–14)

Recent organisation theory has embraced DSM interpretations of classical organisation design concepts. Raveendran et al (2020) credit the above-mentioned Carliss Baldwin (in unpublished work) for formalising Thompson's (1967) pooled, sequential, and reciprocal interdependence in DSM form (see Figure 5). Organisation theorists have also sought to extend the dependency matrix logic beyond positive task interdependence. Most prominently, Worren and Pope (2024) propose a modified partitioning rationale that explicitly accounts for goal conflict among roles or functions. They introduce the rationale for separation as a hard constraint in the matrix representation by "freezing" certain rows to prevent them from being grouped with incompatible others. Figure 6 visualises their theoretic approach, illustrating how additional organisational rationales can be partially incorporated into a matrix framework, citing the DSM literature explicitly.

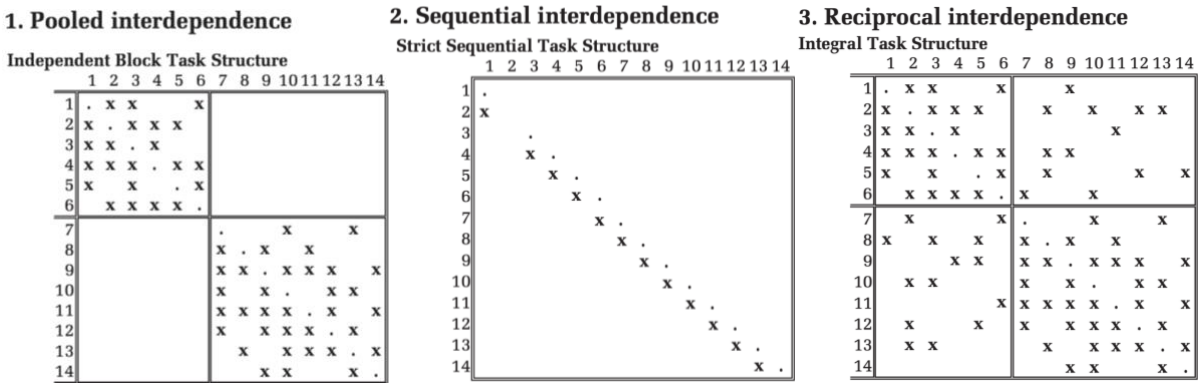


Figure 5 Matrix representation of Thompson's (1967) task interdependence by Carliss Baldwin reproduced in Raveendran et al (2020, p. 838)

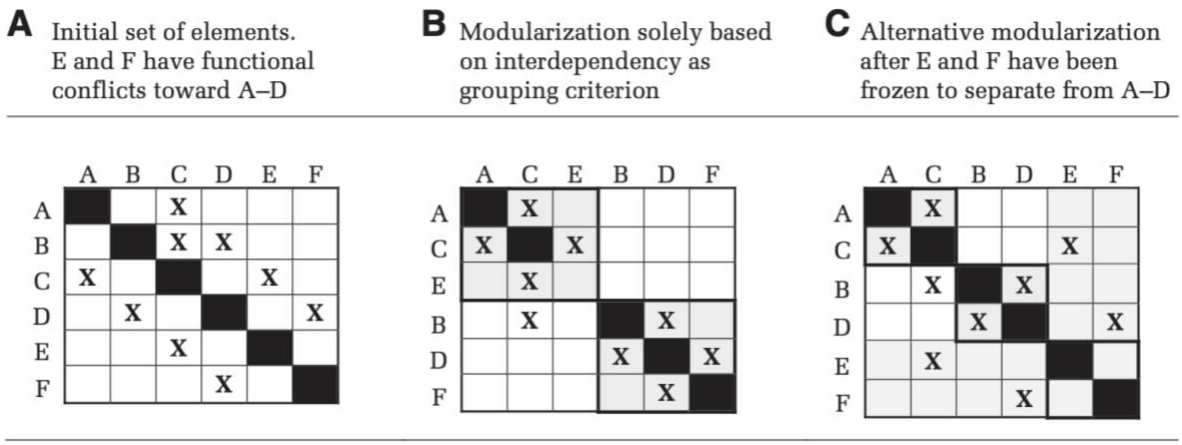


Figure 6 Modified principle for modularisation by Worren & Pope (2024, p. 16)

Empirical applications of these ideas remain limited, however. Studies that operationalise matrix representations of task interdependence with real-world data tend to rely on higher-level proxies – such as input–output tables capturing commodity flows between industrial subsectors that the firm is active in (Zhou, 2013) – rather than direct task-level data. As a result, while matrix-based representations have become influential in computational modelling and theory building, their empirical integration into organisation structure research remains partial, and blockmodeling as a method for detecting clusters has not been picked up.

One notable early exception, as already noted in the introduction of this thesis (see section 1.2), is Nelson's (1986) proposal to use blockmodeling to analyse organisational structure. He later extended this approach by developing relational matrices and cause maps that linked organisational units to critical organisational outcomes (Nelson & Mathews, 1991), similar to the bottom KPI-row in STS's variance matrix (see Figure 1). Despite its conceptual alignment with organisation design theory, this line of work did not gain sustained traction at the time, leaving its potential largely unrealised.

3 Applying blockmodeling to organisation design

Chapter 2 established why the task network — rather than the social interaction network — is the appropriate object of analysis to understand the coordination needs arising from the task environment. This chapter builds the methodological framework for analysing it, working through the three core components of blockmodeling: nodes (3.1), link types (3.2), and criterion function (3.3). Rather than treating these as purely technical choices, each section reviews the options from the perspective of organisation design theory to ground the methodological decisions.

3.1 Nodes: tasks vs people

Organisation design involves the grouping of tasks into units and the allocation of people to those units and tasks. In network terms, both tasks (or aggregates like activities) and people (or aggregates like teams or departments) can be considered as nodes. This section explores how these nodes can be represented in *one-mode*, *two-mode*, *linked* and *multiplex* networks, each offering differential insights for organisational design. The relationships between the nodes (or edges) are discussed in section 3.2.

3.1.1 One-mode networks

One-mode networks consist of a single type of node, where rows and columns represent the same set of entities (Doreian et al., 2005; Scott & Carrington, 2014). In matrix notation, this amounts to a single relation R between the set of n units U, described by a corresponding square and binary matrix $R = [r_{ij}]_{n \times n}$ which jointly determine the network $N = (U, R)$ (Batagelj, Doreian, et al., 1992). This approach simplifies the analysis by focusing on one aspect of the organisation at a time.

- **People-to-people:** A one-mode network of people in organisations focuses on relationships such as reporting structures or team memberships. Each node represents a person, and edges represent relationships like “reports to”, “communicates with”, or “is a member of the same team.” Networks of reporting relationships are ideal for visualizing the organigram or understanding team composition. Figure 7 shows such a network of authority relationships from Doreian et al. (2005, p. 18). The associated blockmodel on the right, shows that the organisation can be viewed as a hierarchy with four levels. Alternatively, worker-to-worker communication patterns can reveal informal networks and information flows within the organisation and identify both “linchpins” and “obstructors” of information (Brusco et al., 2021).

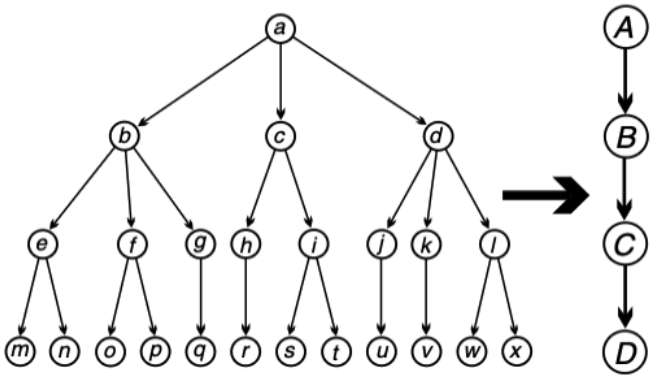


Figure 7 Network and blockmodel of an organisational hierarchy (Doreian et al., 2005, p. 18)

- **Tasks-to-tasks:** Another crucial element in organisation design is the task structure. In a study of the Bank Wiring Room dataset, Homans (1950) as reported in Doreian et al. (2005, p. 41) notes that the relationships between coworkers are not solely defined by their affinity towards each other, or the 'internal system', but also by aspects of the 'external system' such as the layout of the workspace and the sequential nature of the tasks. In a one-mode network of tasks, each node represents a task, and edges represent interdependencies between tasks. These relationships between tasks are a purely technical (non-social) matter, reflecting how one tasks feed in or depends on another one. This representation is particularly useful for visualizing the task structure of the organisation and identifying natural clusters of work activities. Figure 8 shows an example of such a task-to-task matrix. The network can be visualised both as a matrix (on the left) and as a directed graph or network diagram (on the right).

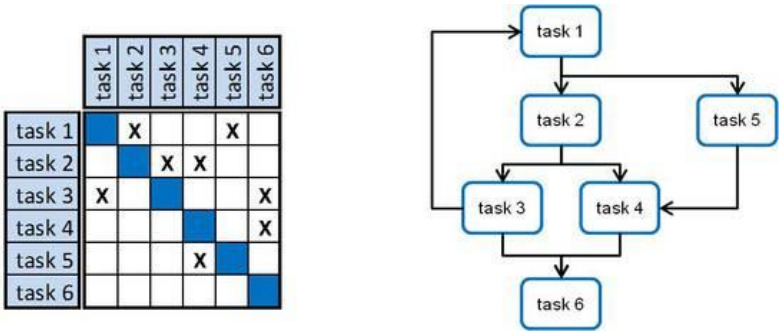


Figure 8 Unimode task-to-task network
Source: Design and Structure Matrix (DSM) (2019)

People or tasks: a social vs a technical matter. Blockmodeling of a people-to-people network addresses mainly HR questions, such as visualising formal reporting relationships or identifying informal networks of communication and mentorship. Blockmodeling of a task-to-task network is an operational matter rather than a people matter. It answers questions about which tasks should be grouped together (e.g., within a single department or team) based on their interdependencies, without yet considering any human resource allocation to those units. For example, tasks that are highly interdependent may be grouped to minimize coordination costs, while tasks with low interdependence can be separated into different structural segments.

3.1.2 Two mode networks

Two-mode networks consist of two distinct types of nodes, where rows and columns represent two different sets of units, U_1 and U_2, and the relationships are only between different types of nodes (Everett & Borgatti, 2020). The matrix is no longer square but is now rectangular as the two sets may have different sizes, n_1 and n_2. The network is defined by $N = (U_1, U_2, R)$ with the relationship $R = [r_{ij}]_{n_1 \times n_2}$ (Doreian et al., 2004). Common examples of two-mode networks include people attending events, directors serving on boards or supreme court justices voting on cases (Brusco et al., 2021). In the context of organisation design, relevant sets of units can be tasks, skills, people and units.

- **Tasks-to-people:** When one set of nodes represents tasks and the other people, then the links indicate which task is assigned to which person, i.e. the task allocation or job design. This network is particularly valuable for HR applications, such as designing function classifications and reward systems, balancing workloads or supporting career trajectories. Figure 9 shows an example of such a task-to-people matrix, where person 1 holds tasks 1 and 5, person 2 holds task 2, etc. In the below example, each task is held by only one person. However, organisations could build in redundancy in task allocation by providing back-ups for each task.

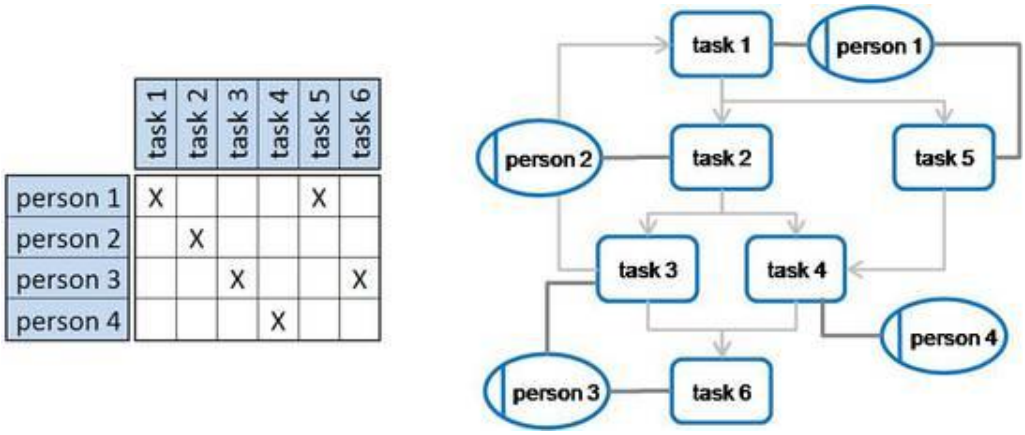


Figure 9 Two-mode people-to-tasks network
Source: Domain Mapping Matrix (DMM) (2019)

- **Skills to people**: Another relevant two-mode network is one where one set of nodes represents skills or competencies, and the other set represents people. This network forms the basis for a competence matrix, a tool for visualizing the skills available in a team. The matrix shows for each team member the level of mastery of a specific skill with a score (e.g. 0-4). A competence matrix helps identify skill gaps, plan training and development initiatives, and ensure that the right skills are available in the team. Figure 10 shows, as an example, that persons 3 and 5 have the widest skill sets in the team, while persons 2 and 4 are specialised in one particular skill and persons 1 is a novice. The matrix also shows that skill 3 has sufficient redundancy, as four out of five team members possess this skill at least at level 2. Skill 4 is in a precarious situation as only one person in the team possesses this skill with sufficient proficiency.

Team member	Person 1	Person 2	Person 3	Person 4	Person 5	Total at least 2
Skill 1	◔	○	●	○	◔	2
Skill 2	○	●	◑	◔	◑	2
Skill 3	◔	◔	◕	●	◕	4
Skill 4	○	○	◔	○	●	1

Figure 10 Competence matrix

3.1.3 Linked: Tasks-to-tasks, tasks-to-people, and people-to-people

Linked (multi-relational) networks are a collection of one-mode networks, where the nodes from different one-mode networks are connected through two-mode networks (Žiberna, 2020). They are composed of one-mode networks for each set (representing ties among nodes of the same type) and two-mode networks that "join" units from different sets. *Multilevel* networks can be considered a special case of such linked networks, when the node sets represent different levels. A common example of multilevel networks is scientific collaboration between researchers and their institutions, through co-authorship at the individual level (level 1), joint project collaboration at the organisational level (level 2) and organisational membership of researchers (cross-level) (Cugmas et al., 2020). By integrating multiple node types, the linked network provides a comprehensive view of the organisation as a whole.

- **Linked network of tasks and people:** A linked approach could consider three types of relationships simultaneously: tasks-to-tasks, people-to-people and tasks-to-people. This allows for a holistic analysis of task interdependencies, task assignments, and team structures. For example, organisations can identify how task groupings (from the tasks-to-tasks network) align

with team structures (from the people-to-people network) and individual assignments (from the tasks-to-people network). This integrated approach supports complex organisational design decisions, such as restructuring departments and optimizing team compositions. Figure 11 shows an example of such a linked matrix of tasks and people, even including additionally a multiplex representation of the people-to-people network: one representing reporting relationships and one representing project team memberships. The multiplex network is further discussed in section 3.2, as it relates to the multiplicity of ties.

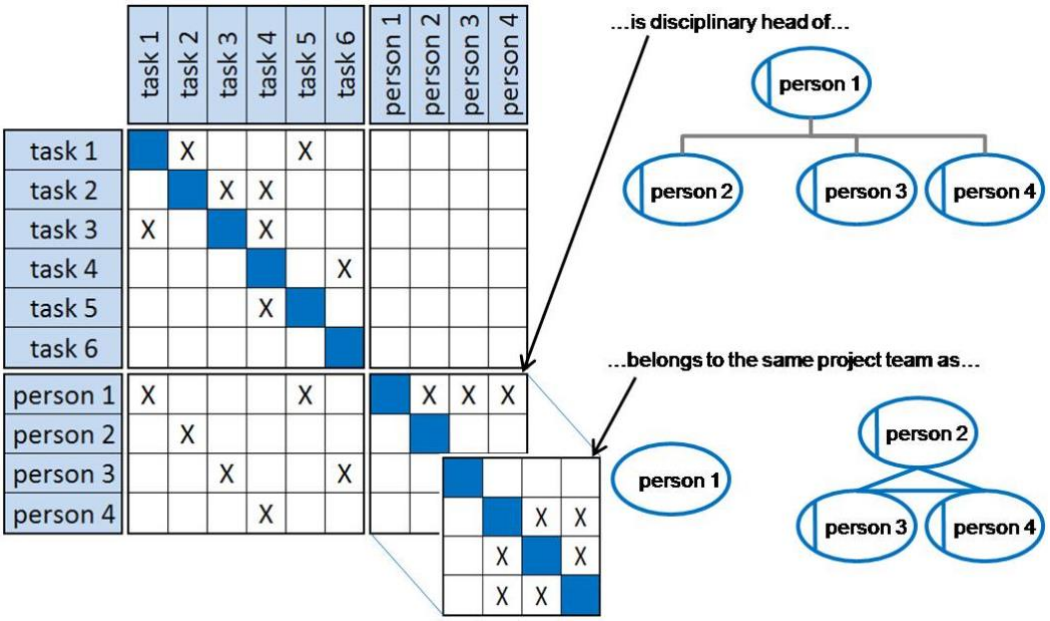


Figure 11 Multilevel network of people and tasks
Source: Multiple-Domain Matrix (MDM) (2019)

- **PCANS model (Precedence, Commitment, Assignment, Network, and Skill):** Both Borgatti & Halgin (2014) and Žiberna (2020) refer to a multilevel approach to organisation design developed by Krackhardt and Carley (1998). Their PCANS model relates persons, tasks, and resources to each other in different one-mode and two-mode combinations. **P**recedence documents sequential interdependencies or temporal ordering between *tasks* (one-mode), **C**ommitment captures which *resources* have been committed to which *tasks* (two-mode), **A**ssignment documents the assignment of *people* to *tasks* (two-mode), **N**etwork captures the relationships between *people* (one-mode) and **S**kill documents which *skills* are held by which *people* (two-mode). By multiplying these relational matrices, a wide range of structures can be described to analyse or control organisations. As the authors write (Krackhardt & Carley, 1998, pp. 115–116):

  > For example, the P relation by itself describes sequential interdependence among tasks. By pre-multiplying P by A (the assignment of individuals to those tasks), we find which tasks those people are dependent on having done before they can do their own tasks. By post-multiplying this word [sic] by A' (APA'), we find the particular personnel they are dependent on to complete tasks before they can complete their own tasks.

Choice of nodes:

- **Relevant options for organisation design**: Blockmodeling offers a range of relevant options for organisation design: one-mode task-to-task networks for partitioning clusters of internally dense activities, one-mode people-to-people networks for visualizing organigrams, and two-mode task-to-people networks for HR applications such as competency matrices, and development plans. Multilevel approaches combining all the above allow for integrated, holistic analysis. Adding resources to tasks and people allows for an even more advanced multilevel network.
- **Choice for MVP in thesis**: For the MVP in this thesis, the focus is on **one-mode task-to-task networks.** As discussed in section 2.2, such a network establishes a baseline understanding of task dependencies and coordination needs. Future research can build on this foundation to explore multimode and linked approaches, expanding the framework's applicability to more complex organisational design challenges.

3.2 Links: types of task dependency

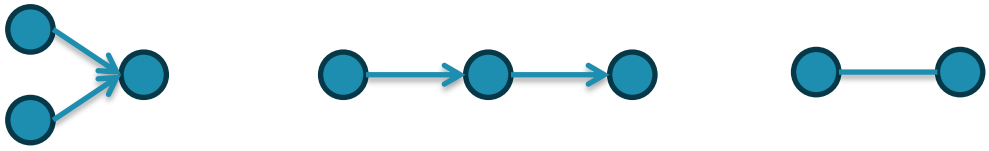
As we focus on the task-to-task network, the relationship between tasks becomes the core relational object. In organisation design, this relationship is referred to as task interdependence, which is defined as "*two tasks are interdependent if the value generated from performing each is different when the other task is performed versus when it is not*" (Puranam et al., 2012, p. 421). In blockmodeling terms, task interdependence can be directed or undirected, have different facets or types, and be signed or valued.

3.2.1 Relevance of blockmodeling link types for organisation design

3.2.1.1 Directed or undirected links

Blockmodeling allows for links between the nodes to be directed (one node sends the link, the other receives it) or undirected (Batagelj, 1997; Doreian et al., 2005). Undirected ties, formally called edges, join two nodes without a specific direction; the relationship is considered mutual or symmetric. Directed ties, formally called arcs, move from an initial node to a terminal node, representing asymmetric relationships. Common examples of undirected ties are marriage ties between families or co-authoring a scientific paper (Doreian et al., 2005; Scott & Carrington, 2014).

Organisation design also makes a directionality distinction in task interdependence. Thompson (1967), for example, distinguishes three primary types: pooled, sequential, and reciprocal. Pooled interdependence (Figure 12a) occurs when tasks contribute to a common outcome but do not directly interact – in this case two nodes send a link to a third one. Sequential interdependence (Figure 12b) involves tasks that must be performed in a specific order, where the output of one task is the input for the next – in this case one node sends a link to another. Reciprocal interdependence (Figure 12c) requires mutual adjustment and continuous interaction between tasks – in this case two nodes are connected in a symmetric way. These types can be represented as either directed (e.g., pooled or sequential) or undirected (e.g., reciprocal) links in a network, depending on the nature of the dependency and the flow of work between tasks.



(a) Pooled (b) Sequential (c) Reciprocal

Figure 12 Types of interdependence (directed / undirected) according to Thompson (1967)

3.2.1.2 Multiplex links

In **blockmodeling** multiplexity refers to the presence of multiple types of relations defined over the same set of nodes. It can be conceived as a series of matrices (or 'slices'), each of which describes a single type of tie among all pairs of actors (Hanneman & Riddle, 2014). It allows for a layered analysis where actors are connected in multiple ways simultaneously. A common example includes both the formal reporting relationships (the formal structure) and the informal communications or advice-seeking relationships (the informal structure) among the same set of employees (Cugmas et al., 2021).

In **organisation design**, Task interdependence has been further disaggregated into different types of dependence. Raveendran, Silvestri, and Gulati (2020) distinguish three types: task, goal, and knowledge interdependence. Task interdependence refers to the workflow dependencies between tasks, while goal interdependence concerns the alignment of objectives across tasks or units. Knowledge interdependence highlights the need for shared expertise to complete tasks effectively. In a network, these can be represented as multiplex links, where multiple types of relationships coexist between the same nodes, each layer capturing a different dimension of interdependence.

The Role of Task, Goal, and Knowledge Interdependence in Organization Design

	Task Interdependence	Goal Interdependence	Knowledge Interdependence
Ethos Our proposed definition	Who does what? Two tasks are interdependent if the value generated from performing each is different when the other task is performed versus when it is not. (cf. Puranam et al., 2012: 421)	Who wants what? Two agents are goal interdependent if they share a common goal, whether or not they actually work together.(adapted from Mitchell & Silver, 1990: 186)	Who knows what? Two agents are knowledge interdependent if the value they could generate from combining their knowledge differs from the value they could obtain from applying their knowledge separately.

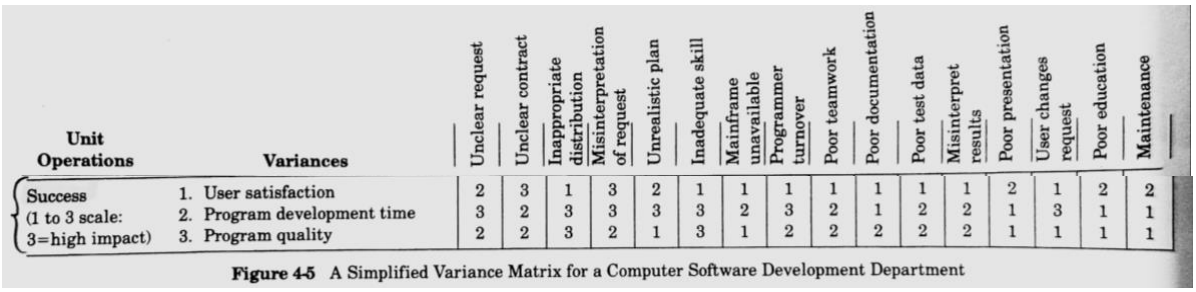
Figure 13 Extract from Table 4 in Raveendran et al. (2020)

3.2.1.3 Binary or valued links

In **blockmodeling** binary links are the simplest form, where a tie either exists (1) or does not exist (0) (Hanneman & Riddle, 2014). Valued or weighted links assign a numerical magnitude to represent the intensity, frequency, or strength of the relationship, typically measured on at least an interval scale, such as a Likert-type 1 to 5 rating scale (Nordlund & Žiberna, 2020). The strength of the tie can also be deduced as direct measures of the frequency of interactions, such as the number of e-mail exchanges between people or the number of note-borrowing between students (in one-mode networks) or the number of co-authorships on papers, co-memberships in organisations or co-attendance to events (in two-mode networks).

In **organisation design**, an example of valued ties can be found in the variance matrix of Pasmore (1988). The one-mode (variance-to-variance) section of the matrix documents in a binary way how variations – or deviations from the norm – in one task propagate to following tasks (see Figure 1 in the introduction). The two-mode (variance-to-outcome) section of the variance matrix (see Figure 14) documents how much the outcome of the whole work process – such as customer satisfaction,

throughput time or product quality – is influenced by these variations in the tasks. This impact of task variation on the success factors is measured on a 1-3 scale, where 1 is low impact and 3 is high impact. This approach can be translated into network terms as valued links, where the strength or weight of a link reflects the intensity or criticality of the dependence. Binary links, by contrast, only indicate the presence or absence of a relationship, without quantifying its importance.



Unit Operations	Variances	Unclear request	Unclear contract	Inappropriate distribution	Misinterpretation of request	Unrealistic plan	Inadequate skill	Mainframe unavailable	Programmer turnover	Poor teamwork	Poor documentation	Poor test data	Misinterpret results	Poor presentation	User changes request	Poor education	Maintenance
Success	1. User satisfaction	2	3	1	3	2	1	1	1	1	1	1	1	2	1	2	2
(1 to 3 scale:	2. Program development time	3	2	3	3	3	3	2	3	2	1	2	2	1	3	1	1
3=high impact)	3. Program quality	2	2	3	2	1	3	1	2	2	2	2	2	1	1	1	1

Figure 4-5 A Simplified Variance Matrix for a Computer Software Development Department

Figure 14 Bottom rows of variance matrix in Pasmore (1988, p. 75)

3.2.1.4 Signed or unsigned links

In **blockmodeling** the most common links are unsigned, which only represent the presence, absence (or strength, in case of valued) of a tie using non-negative values. Signed links, instead, incorporate sentiment, using positive values (+1) for "pro" relations (liking, trust, alliance) and negative values (−1) for "anti" relations (disliking, distrust, conflict) (Brusco et al., 2021; Traag et al., 2020). Common examples for unsigned links include friendship ties or communication counts, while common examples of signed links are the co-existence of liking and disliking ties between children in classes or trusting and distrusting ties between workers in organisations.

In **organisation design**, the principle underlying signed links is relevant to the concept of functional conflict (Worren & Pope, 2024). Control or audit functions may have genuine workflow dependencies with the products or processes they oversee but at the same time have conflicting goals. While operations aim to maximise throughput, control aims to maximise defect detection. Grouping control functions with operations would reduce coordination costs but undermine organisational functioning. Worren and Pope resolve this by freezing certain node pairs — a hard constraint that prohibits them from being assigned to the same cluster as the products they control (see Figure 6 in the introduction).

Another, second, operationalisation could be to introduce positive links for interdependencies that require integration and negative links for functional conflicts that require separation in a single layer. However, such an implementation would limit the possibility of having both positive task dependence and negative goal conflict at the same time.

This thesis thus takes a third approach. Rather than imposing a hard constraint outside the criterion function, or signing the links in one layer, goal conflict is modelled as a separate layer with null-diagonal ideal blocks. This keeps the separation logic inside the optimisation: the weight assigned to the conflict layer relative to the interdependence layer can be adjusted to reflect strategic priorities, allowing for a genuine trade-off between integration and separation rather than treating separation as an absolute prohibition.

3.2.2 Common grouping or separating rationales in organisation design

The above four *technical* link types form the representational vocabulary of blockmodeling. Organisation design theory now determines the *substantive* link types relevant for grouping or separating tasks. Three theoretical rationales each map directly onto a specific set of these link properties.

3.2.2.1 Specialisation by grouping functionally similar tasks (functional dependency)

The classical starting point for designing an organisation structure lies in the principle of **specialisation of expertise**, following the division of labour. This principle proposes to group together similar tasks – tasks requiring the same actions or resources. Adam Smith's account of specialisation in *The Wealth of Nations* (1776) is illustrated by the famous pin factory example. Earlier craft production required the worker to complete all sequential tasks for manufacturing a whole pin, requiring a wider skill set. Decomposing production into a sequence of narrowly defined tasks and assigning each worker to a specific operation, instead, allowed workers to build specialised skills for this task.

This division of labour dramatically increased output relative to craft production. Smith attributed these efficiency gains to learning through repetition, time saved from task switching, and the possibility of task-specific improvements. This logic of grouping similar activities to exploit scale efficiencies has profoundly shaped the modern firm. Similarity can be defined as activities requiring the same set of resources, be it cognitive resources (like information, skill, knowledge or expertise) or material resources (like a machine or a tool). Functional departments – human resources, information technology, finance, procurement, and other specialised units – concentrate such similar tasks and competencies to enhance efficiency, consistency, and expertise.

This logic was later formalised and radicalised by **scientific management**. Taylor (1911) reframed specialisation not merely as a spontaneous outcome of learning or skill differentiation, but as the result of deliberate managerial task design. Through systematic time-and-motion studies, work was decomposed into its smallest constituent elements, and for each task a single optimal method—the "one best way"—was identified by the manager or consultant. Scale efficiencies were generated through standardisation and repetition, but also through the separation of conception from execution, with knowledge and control residing in management rather than in workers. From this perspective, grouping similar tasks is efficient not only because it facilitates learning and expertise, but because it enables uniform work processes, tighter coordination, and predictable output.

However, as organisation theorists have long recognised, the benefits of such grouping come with significant trade-offs. Functional specialisation increases the need for coordination across departments, because organisational goals typically require joint contributions from multiple functions. As March and Simon (1958) and later Mintzberg (1979) observed, the more an organisation decomposes work into specialised units, the greater the demand for integrating mechanisms – rules, routines, supervision, planning, mutual adjustment, and communication channels. In other words, grouping by similarity can unintentionally create substantial interdependence across units, imposing large coordination burdens on the organisation as a whole. Modern organisations often experience this tension of working in silo's acutely: while specialised units may excel at their narrow tasks, cross-functional collaboration becomes cumbersome and costly.

This classical perspective therefore explains one rationale for grouping – referred to as **functional dependence** in this thesis – but cannot fully address how organisations should structure themselves when coordination needs point in other directions.

3.2.2.2 Coordination by grouping tasks across end-to-end workflows (workflow dependency)

A second foundational rationale for organisational grouping stems from the work of Thompson (1967) and the information-processing tradition. From this perspective, the primary design problem is not achieving economies of scale but **managing workflow interdependence**. When the performance of one activity depends on timely and accurate inputs from another, organising them within the same unit reduces the need for costly cross-unit coordination. Thompson distinguished different types of such interdependence, namely pooled, sequential and reciprocal interdependence. His core design implication is that as interdependence becomes more reciprocal, organisations must rely on richer

lateral coordination mechanisms, making unit boundaries and grouping choices central to managing coordination costs. Galbraith (1974) similarly argued that organisational structure must balance the *need* for information processing – created by task interdependence – with the organisation's *capacity* to process information through formal and informal coordination mechanisms.

This view has strongly influenced both organisational theory and engineering approaches to **managing complex systems**. The modularity literature and the design structure matrix (DSM) tradition emphasise grouping tightly interdependent elements into modules to create "nearly decomposable'' architectures (Baldwin & Clark, 2000; Eppinger & Browning, 2012; Simon, 1962). Such grouping reduces the volume of cross-module interactions, keeping within-module interactions dense. In organisational contexts, modular units become self-contained structures that can operate with minimal need for integration elsewhere in the organisation.

Contemporary scholarship underscores the centrality of interdependence in the micro-foundations of organisational design (Puranam, 2018). Raveendran, Silvestri and Gulati (2020) distinguish between task, goal and knowledge interdependence, each creating distinct coordination requirements. The ability to diagnose interdependence patterns is thus essential for structuring teams, departments and processes in ways that support efficient information processing.

From this second perspective, a core rationale for grouping is **workflow interdependence**: activities that rely on or follow each other should be co-located to minimise coordination costs. Yet this logic alone cannot account for situations in which activities are tightly connected but ought nevertheless to be separated.

3.2.2.3 Avoiding conflict by separating incompatible roles

A recent strand of organisation design theory highlights that some activities, despite being closely connected or workflow interdependent, should nevertheless *not* be grouped. Worren and Pope (2024) argue that roles may hold functional goals that are inherently incompatible, such that placing them within the same unit reduces effectiveness, undermines accountability, or increases the likelihood of biased decision-making. They describe organisations as "connected but conflicted": even though units depend on one another, they also pursue goals that sometimes cannot be jointly optimised.

Classic examples include separating audit, risk assessment, or quality assurance from the roles that they are auditing, assessing or assuring. In such cases, grouping the control with the operational function would increase coordination ease but at the expense of quality, compliance or long-term organisational functioning. The presence of conflicting goals therefore constitutes a third structural rationale, distinct from both similarity and interdependence. It calls not for grouping, but for **task separation**.

This separation perspective introduces a new form of design tension. Organisations must simultaneously manage interdependence (which calls for integration) and conflicts of purpose (which call for separation), while keeping an eye on scale efficiencies. The result is an architectural balancing act: integrating where coordination or scale is essential, separating where incompatible goals must be kept at arm's length.

Choice of links:

- **Relevant options for organisation design**: The blockmodeling framework offers a range of options for representing interdependence between work tasks: directed or undirected links for capturing the flow of work, multiplex links for distinguishing task, goal, and

knowledge interdependencies, signed links for distinguishing alignment or conflict, and binary or valued links for quantifying the intensity of the interdependence.

- **Choice for MVP in thesis**: For the MVP in this thesis, the focus will be on **multiplex links** – but excluding valued, signed and directed links – to establish a foundational understanding of three types of linkages: two linkages based on grouping rationales, namely functional and workflow dependencies, and one linkage based on a separation rationale, namely goal conflict. Future research can explore the richer representations offered by valued and directed links.

3.3 Criterion function: getting as close to ideal as possible

In blockmodeling, the criterion function is the fundamental instrument used to evaluate how well an empirical partition fits a theoretical model. Formally, it is a measure that quantifies the total inconsistency (also called error or tension) between the observed network ties and an "ideal" block structure (Batagelj, 1997; Doreian et al., 2005). How this criterion function is derived and optimized determines the basic typology of blockmodeling. The allowed block types further distinguish between the theoretical foundations of the equivalence types used to justify the clusters.

3.3.1 Basic typologies of blockmodeling

- **Deterministic vs. stochastic.** Deterministic blockmodeling does not rely on an underlying statistical model – it seeks a partition that minimizes a criterion function based on fixed rules for permitted block types (Brusco et al., 2021). Stochastic blockmodeling, in contrast, uses generative probabilistic models grounded in random graph principles. It infers node classes by finding the model parameters that have the highest likelihood of generating the observed data (Peixoto, 2020). This approach uniquely allows to distinguish genuine structure from randomness in the data.

  In **organisation design**, most authors treat the links between nodes (i.e. interdependencies between tasks) as reflecting a true underlying task structure that originates in the production process. However, these interdependences are rarely actually observed, and instead are often inferred indirectly. They can be inferred from production or process data, or through interviews, focus groups or workshops aimed at explicating the interdependencies. However, people's perceptions of task interdependence can deviate from the actual latent ones by as much as 30%, and these perceptual errors are almost entirely in the direction of *underestimating* the level of interdependence (Sherman & Keller, 2011). If the possibility of randomness entering the data collection process should be considered, then stochastic blockmodeling might be preferred.

  In **this thesis**, task interdependence is deduced from production data, such as the routing of products over machines, functional characteristics of the machines and product characteristics from the production catalogue. As such, noise in the data is assumed to be small and blockmodeling is implemented deterministically.

- **Direct vs. indirect.** The direct approach works directly with the network data to minimize the criterion function, by counting the raw inconsistencies between the observed network and the ideal block structure (Batagelj, Ferligoj, et al., 1992). The indirect approach, in contrast, first transforms network data into a (dis)similarity matrix (e.g., using Euclidean distance or

correlations) and then applies standard clustering algorithms like Ward's method (Doreian et al., 2005). The direct approach is generally considered superior because the measure of fit is built into the optimization process (i.e. the number of inconsistencies) (Doreian et al., 2005).

For **organisation design**, the direct approach has the benefit of allowing to quantify how many task interdependencies are left between the clusters instead of within the clusters. Ideally, the within-cluster density is higher than the between-cluster density so that the task interdependencies can be managed within the cluster (i.e. within the organisational unit or team). By counting the remaining interdependencies between the clusters, the organisation designer can point towards the need for additional integrative mechanisms such as cross-unit meeting structures, standard interfaces or service-level agreements. This paper implements direct blockmodeling for that reason.

- **Inductive vs. deductive.** Inductive (or exploratory) approaches start without pre-conceived notions of the structure to be found in the network. A set of allowed block types is specified by the researcher (reflecting some form of theoretical equivalence between the nodes, see section 3.3.2) and clusters are sought that are as consistent as possible with these block types (Doreian et al., 2005). Deductive (or confirmatory) approaches use prior theoretical or empirical knowledge to *prespecify* the location and type of blocks in an image matrix (Batagelj et al., 1998). Deductive approaches may also include *constraints*, such as some nodes always or never belonging to the same cluster, or the maximum number of nodes that may belong to a cluster (Doreian et al., 2005, p. 235)

  For **organisation design**, the potential of deductive approaches is large. Pre-specification in a task-to-task network may build on the need for modular organisations – with high tie densities on the diagonal blocks and low tie densities on the off-diagonal ones. Pre-specification in a people-to-people network may build the need for unambiguous reporting lines (i.e. strictly hierarchical block models). Separation of functional conflict (see 3.2.1.4) can be achieved both through pre-specification – by specifying on-diagonal null blocks – or through constraints – by prohibiting some units from belonging to the same cluster. Finally, team sizes or workloads may be managed by constraining the maximum number of nodes that may belong to a cluster. This paper implements deductive blockmodeling for both the grouping rationales (complete on-diagonal blocks) and the separation rationales (null on-diagonal blocks).

3.3.2 Allowed block types

As discussed above, the criterion function in blockmodeling evaluates how closely an empirical network matches an ideal structural pattern by measuring inconsistencies between observed ties and ideal block types. The set of allowed block types determines the specific notion of equivalence being applied to the nodes in the network (Doreian et al., 2005). 'Equivalence' here means that the nodes are similar to each other, in a way to be further specified below, and that this equivalence is the basis for detecting clusters or groups.

- **Structural equivalence.** Structural equivalence is the most stringent form of equivalence. It implies that two nodes are structurally equivalent if they are connected to the rest of the network in identical ways. Essentially, they have the same neighbours. In order for nodes A and B to be clustered together, if node A has a tie to node C, node B must also have a tie to node C. Nodes that are connected in this way are "structurally indistinguishable" from one another and occupy a common structural position. In Ferligoj et al. (2014) a set of structurally equivalent actors is called a *position*. The permitted block types associated with structural equivalence are the Null (all 0s) and Complete (all 1s) blocks.

- **Regular equivalence.** Regular equivalence is a generalization of structural equivalence, offering a less restrictive way to define similar nodes. Instead of being equivalently connected to the exact same other nodes, they can be *equivalently connected* to *equivalent others* (Doreian et al., 2005). Regularly equivalent nodes do not need to be connected to the *same* neighbours, but to neighbours playing the *same roles* in the network. Ferligoj et al. (2014) provide the intuition that, while a *position* is a location in some social structure, a *role* has a set of expected behaviours corresponding to the location. For example, while parents maybe structurally connected to different children (and thus occupy different *positions* in the network), they may carry the same behavioural roles towards their children (and thus occupy similar *roles* in the network). Under regular equivalence, permitted block types are expanded to include regular (also called 1-covered) blocks (Doreian et al., 2005). A regular block requires that each row and each column contain at least one tie.

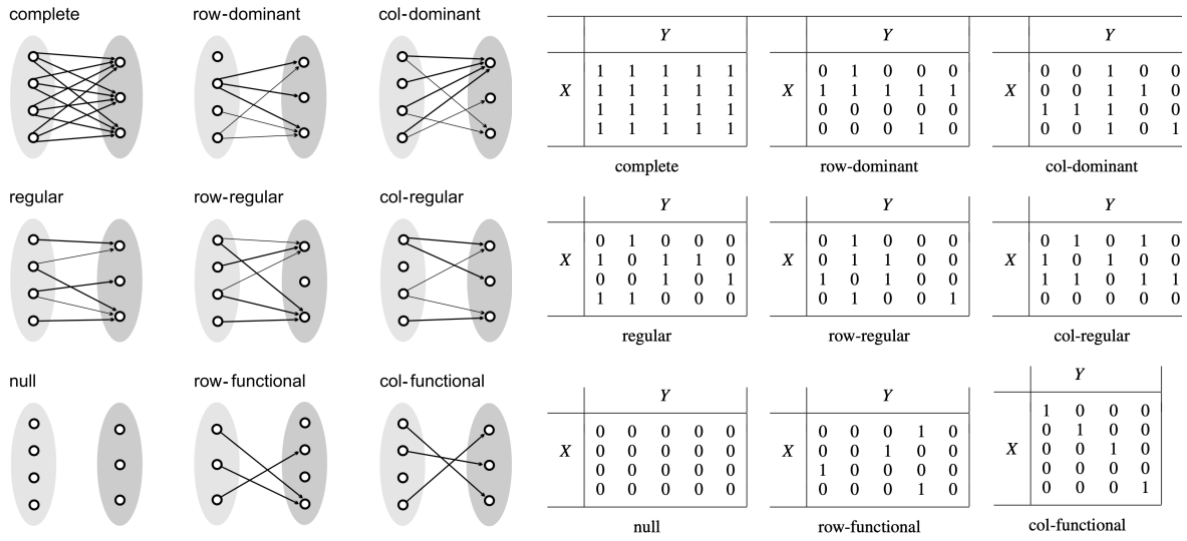
- **Generalized blockmodeling**. Generalized blockmodeling allows the analyst to expand the set of ideal blocks beyond the classic types to better capture specific organizational or social structures (Doreian et al., 2005), see Figure 15.

    - **Row-regular and column-regular blocks** are weakened versions of regular equivalence. In a row-regular block every node in the source cluster sends at least one tie to the target cluster, but not every node in the target cluster necessarily receives one. In a column-regular block, every node in the target cluster receives at least one tie, but not everyone in the source cluster necessarily sends one. These types of blocks represent unidirectional dependency. For example in Nordlund (2016)'s analysis of a journal citation network, the "row-regular" block on the upper-right of the matrix indicates that all core journals have prominent citations to at least one peripheral journal, although not all peripheral journals are prominently cited by core journals.

    - **Row-dominant and column-dominant blocks** identify highly influential or central units. In a row-dominant block, there is at least one node in the source cluster who has a tie to every single node in the target cluster, while in the column-dominant block, there is at least one node in the target cluster who receives ties from every single node in the source cluster. These types of blocks represent control or high visibility nodes (Doreian et al., 2005). A "row-dominant" block implies a "controller" who can reach everyone in another group. A "column-dominant" tie to a cluster indicates a "sociometric star" that everyone in another group recognizes or cites.

    - **Row-functional and column-functional blocks** represent sparse, unique mappings between clusters. In a row-functional block, each node in the source cluster has a tie to exactly one node in the target cluster, while in the column-functional block each node in the target cluster is linked to by exactly one node in the source cluster. These block types are useful for modelling specialized niches or sparse citation patterns where each "consumer" relies on a single "producer" for specific resources (Doreian et al., 2005).

    - **Symmetric blocks** are used for diagonal blocks where actors within a group have reciprocal, mutual ties, but the block is not necessarily complete. This captures "clique-like" behaviour with cohesive subgroups on the diagonal where internal relationships are balanced and symmetric, but not complete.



(a) Graph representation (b) Matrix representation

Figure 15 Nine types of connections between clusters from (Doreian et al., 2005, p. 212)

3.3.3 Handling different ties in the criterion function

Beyond the above variations, the criterion function can further be adapted based on the nature of the links and nodes:

- For *directed* networks, links are sent from an initial to a terminal cluster, and the criterion function accounts for this asymmetry by assessing row-wise and column-wise patterns separately.

- For *signed* networks, based on Structural Balance Theory, the implied blockmodel has positive blocks on the diagonal and negative blocks off the diagonal (Traag et al., 2020). The criterion function then counts "imbalanced" ties – specifically positive ties in negative blocks and negative ties in positive blocks.

- *Valued* networks can be handled through several distinct direct and indirect blockmodeling frameworks (Žiberna, 2007, 2008, 2009): (1) Dichotomization entails reducing valued data to binary form using a simple threshold; (2) Valued blockmodeling measures inconsistencies as deviations from either 0 (when a tie should not exist) or a specified threshold *m* (Žiberna, 2007); (3) The deviational approach counts deviations from expected values derived from the distribution of total in- and outdegrees for each actor (Nordlund, 2016); (4) The homogeneity approach minimizes within-block variability – typically through the sum of squared deviations from the mean – to identify blocks where tie values are as uniform as possible (Žiberna, 2007); (5) Adaptive weighted correlation-based blockmodeling replaces classical inconsistency-counting with a goodness-of-fit function that allows for the analysis of valued networks without any form of dichotomization or data transformation (Nordlund, 2020).

- For *multiplex* networks (representing multiple sets of relations between the same sets of nodes), the global criterion function is typically calculated as a weighted sum of the criterion functions for each individual relation.

- In *multimode* networks (representing relations between two sets of nodes), the rows and columns are partitioned simultaneously but differently to fit rectangular matrices (Everett & Borgatti, 2020).

- In *linked* and *multilevel* networks, the multiple node sets (or levels), as well as the cross-set (or cross-level) relations, are first joined together in one large network that contains all ties between the nodes from all sets. This large network is then partitioned using traditional blockmodeling techniques under the restriction that nodes from different sets cannot be placed in the same cluster (Žiberna, 2014).

3.3.4 Aligning structure with strategy: weighting the criterion

Understanding the rationales for grouping or separating tasks (i.e. the multiplex types of links between tasks) still leaves open the question of which one to pick or how to weigh them against each other (i.e. how to construct the criterion function). **Contingency theory** provides an answer: the chosen structure should support the pursuit of the organisation's strategic goals (Chandler, 1962). Different competitive strategies generate different coordination needs or information-processing requirements, and structure should provide the capacity to meet them (Galbraith, 1974; Tushman & Nadler, 1978). Low-cost or mass-production strategies call for functional specialisation and efficiency, pointing towards higher weights on grouping tasks that share similar resources or expertise. Innovation-oriented or small-batch strategies require lateral coordination across workflow dependencies. Risk and compliance contexts require separation of conflicting roles.

Configurational theory translates the strategy-structure fit into recognisable types or configurations. Miles and Snow's (1978) typology illustrates this logic directly. *Prospectors* compete through continuous innovation and market exploration: their primary coordination challenge is lateral, integrating knowledge across functions rather than within stable functional specialisms. *Defenders* on the other hand occupy a narrow, stable domain and compete on efficiency and cost; the dominant problem is building scale within established functional areas of expertise. Weighting the multiplex layers in the criterion function should therefore reflect the main coordination problem: *Prospectors* will prioritise grouping workflow dependencies (supporting end-to-end innovation-oriented cross-functional groupings); *Defenders* will put more weight on resource-similarity links (supporting functional grouping, standardisation and efficiency).

Two qualifications apply. The Resource-Based View (Barney, 1991) and Knowledge-Based View (Grant, 1996) of the firm suggest that structure can itself be a source of competitive advantage, making the relationship between strategy and structure bidirectional rather than strictly one-directional. And as discussed in section 2.2, institutional theory (DiMaggio & Powell, 1983) notes that structures are sometimes adopted for legitimacy rather than strategic fit. While these perspectives explain why strategic orientations may not always drive observed structures in practice, they do not undermine the principle of weighting different types of task links in the criterion function when organisations seek to fit structure and strategy.

Choice of criterion function:

- **Relevant options for organisation design**: The blockmodeling framework offers a range of relevant options for organisation design within the criterion function: deterministic vs. stochastic, direct vs indirect, and inductive vs. deductive approaches. The allowed block

types further distinguish between structural, regular, and generalized equivalence, each allowing to specify their own underlying rational (or equivalence type) for creating clusters.

- **Choice for MVP in thesis**: For the MVP in this thesis, the focus will be on **deterministic, direct, and deductive blockmodeling**, using generalized blockmodeling to adapt the criterion function for multiplex ties. This approach provides a robust foundation for analysing task interdependencies and goal conflicts, while future research can explore stochastic blockmodeling (to account for measurement error) and different ways for handling valued or signed networks.

4 Implementation in R and simulation study

This section develops the analytical apparatus in two steps. Section 4.1 describes the R implementation: how the multiplex task network is constructed, how the block-type pre-specification encodes each organisational rationale, and how the optimisation is run. Sections 4.2 through 4.3.3 test this implementation on a hypothetical production process with a known structure, verifying that the algorithm recovers the expected organisational forms before the method is applied to empirical data in section 5.

4.1 Implementation in R

The analysis uses the `blockmodeling` package in R (Matjašič et al., 2020), which implements generalized blockmodeling for binary and valued networks. The package builds on the theoretical framework of Doreian, Batagelj & Ferligoj (2005) for generalized blockmodeling and Žiberna (2007) for its extension to valued ties. The approach applied here is deterministic, direct, and deductive. It is *deterministic* in that it minimises a criterion function counting inconsistencies rather than fitting a probabilistic model; *direct* in that it works on the observed relational matrices without first computing pairwise similarity scores; and *deductive* in that the ideal block structure is pre-specified from theory before any fitting takes place.

Link types. The input is a multiplex network represented as a three-dimensional array M of dimension $n \times n \times 3$, where n is the number of tasks. A task is defined as an operation of a machine on a product. Each layer is a binary, undirected, zero-diagonal matrix:

- M_1 captures machine-sharing (or a functional dependency): tasks that require the same machine are connected.
- M_2 captures product-sharing (or a workflow dependency): tasks that follow each other because they act on the same product are connected.
- M_3 captures goal conflict: tasks whose functional objectives are incompatible are connected.

Pre-specified block types. For each layer, the ideal block structure is pre-specified as a four-dimensional array indexed by $[row\ cluster, column\ cluster, layer, type]$. Figure 16 shows the ideal blocks for a hypothetical two-cluster partitioning.

For M_1 and M_2, a `comnul` specification is chosen: all diagonal blocks are `com` (complete), requiring within-cluster ties to be present. All off-diagonal blocks are `nul` (null), encoding the expectation that tasks in different clusters are not interdependent. M_1 and M_2 therefore seek internally dense, externally sparse partitions, so that organisational units assigned to these task clusters should coordinate more intra-unit then inter-unit.

For M_3, a `nuldnc` specification is chosen: all diagonal blocks are `nul` (no within-cluster conflict), and all off-diagonal blocks are `dnc` (do not care), contributing zero error regardless of observed ties. This choice reflects that theory requires conflicting tasks to be separated (i.e. not being a member of the same cluster) but makes no prediction about the pattern of cross-cluster conflict ties.

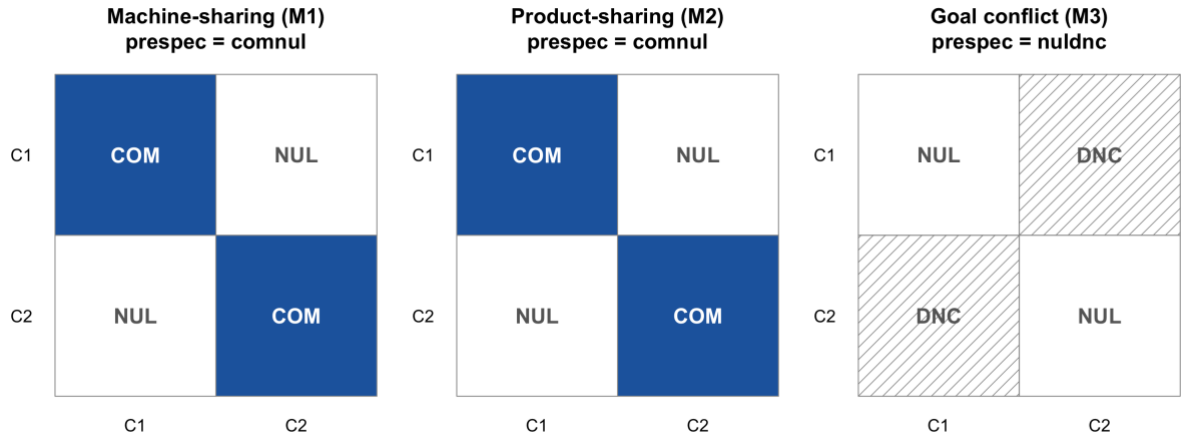


Figure 16 Pre-specified block images for the three layers, illustrated at a uniform k = 2.
Note: COM = complete block (blue); NUL = null block (white); DNC = do-not-care block (white with diagonal hatching). M1 and M2 use the grouping (comnul) structure; M3 uses the separating (nuldnc) structure.

Criterion function. The global criterion function is a weighted sum of three per-layer error functions:

$$F = w_1 \cdot F(M_1) + w_2 \cdot F(M_2) + w_3 \cdot F(M_3)$$

Under binary blockmodeling, a tie is either present or absent. A present tie in a null block contributes one unit of error; an absent tie in a complete block contributes one unit of error. Each per-layer criterion measures the deviation of the observed matrix from its ideal block structure. In some specifications, weights are normalised using $nw = 1 / (n(n-1))$, which expresses each layer's raw violation count as a proportion of the maximum possible violations in an n-node network. At the equal-weight baseline, $w_1 = w_2 = w_3 = nw$, so all three rationales contribute equally. Strategic weighting — assigning higher weights to selected layers — operationalises contingency theory: e.g. an organisation prioritising coordination over workflow dependencies would raise w_2 relative to the others.

Number of clusters. In the simulation study (section 4), the number of clusters k is not fixed in advance. The algorithm is run for $k = 1$ through k_max and the best k^* is selected as:

$$k^* = argmin_k\, F(k)$$

k_max is capped at $n - 1$ as n nodes cannot be partitioned into n or more clusters. A second limitation on the number of clusters is set with `MINGR=3`, or the minimum cluster size in the starting partition having at least 3 nodes. This floor is set to avoid singleton clusters in the final partition. Experimentation showed that setting this floor at 2 quickly leads the blockmodeling algorithm into degenerate solutions with singleton clusters. When multiple values of k share the minimum, the smallest is preferred on grounds of parsimony. In the empirical application (section 5), the large number of tasks makes seeking for the optimal k no longer feasible, so the number of clusters is chosen by the client organisation.

Optimisation. For each value of k, `optRandomParC` performs local search from random starting partitions and retains the solution with the lowest criterion value. No global optimum is guaranteed; multiple restarts reduce sensitivity to starting conditions. The algorithm runs in two stages. First, a k-selection loop fits the model for $k = 1$ through k_max and records the minimum criterion value at each k; the k with the lowest value is selected as k^*. Second, once k^* is known, the model is run again at that k to collect all structurally distinct partitions that achieve the minimum. These two stages place different demands on the number of restarts. In the k-selection loop, restarts only need to produce a reliable estimate of the minimum criterion value at each k — enough to correctly rank k values and

identify k^*. Restarts here are varied across {10, 25, 50}. In the second stage, more restarts are warranted: missing a solution means the reported set is incomplete. Restarts here are varied across {100, 200, 500}. The sensitivity analysis in section 4.2.3 shows that results stabilise well before the upper bound restarts at either stage.

Retrieving all optimal partitions. At k^*, `optRandomParC` is run with `max.iden` set equal to the number of restarts, so the package retains all solutions that tie at the minimum criterion value. Two partitions are considered identical if a relabelling of cluster labels makes them the same. This deduplication is handled by `switch.names = TRUE`, which collapses label-permutation equivalents into a single solution. The setting is valid here because the block structure is symmetric: all diagonal blocks share one type and all off-diagonal blocks share another, so cluster labels carry no positional meaning.

Output. Clusters are reordered for interpretability. Clusters are sorted by machine (in a clustering based on machine-sharing) or by product (in a clustering based on product-sharing). Within each cluster, tasks are sorted alphabetically. The reordered matrices are visualised per layer using `plotMat`, with cluster boundaries drawn explicitly. The code for the simulation (in the form of a Shiny app) is provided in a public GitHub code repository: https://github.com/LMNU/blockmodeling.

4.2 Hypothetical production environment and hypotheses

The simulation is built around a hypothetical production process consisting of four products (A, B, C, D), each passing through four functional machines (1, 2, 3, 4), of which the last one is a quality control inspection (Figure 17). This yields 16 tasks in total — four per product — which form the nodes of the task network.

The assumption in this simulated example is that all tasks requiring the same machine (machine-sharing), are functionally dependent – they share a resource and require similar machine-specific expertise. Tasks acting on the same product (product-sharing), by contrast, follow each other sequentially — the output of one function becomes the input of the next — making them workflow dependent. Finally, for each product, the quality control function is in goal conflict with the manufacturing functions: manufacturing maximises throughput while quality control maximises defect detection, and these objectives cannot be simultaneously satisfied.

These three relationships — machine-sharing across products, product-sharing across functions, and goal conflict between manufacturing and control — are visualised in orange, blue and red lines and arrows in Figure 17 below.

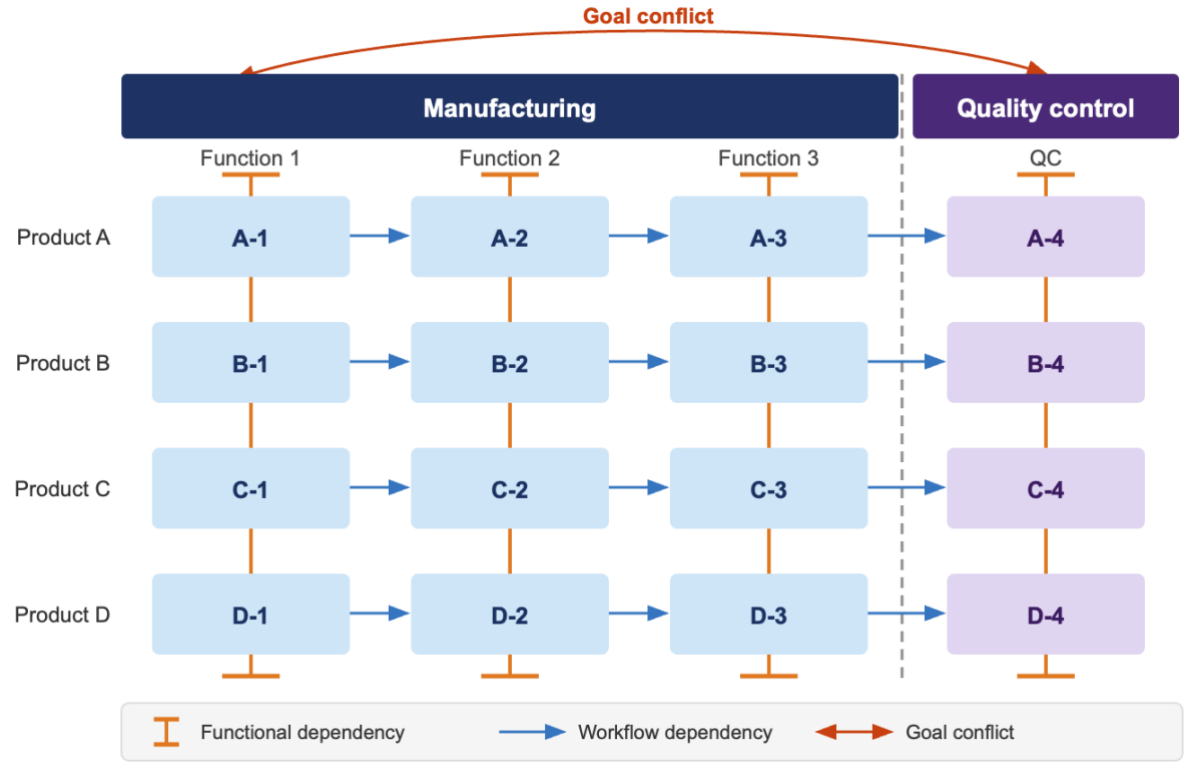


Figure 17 Hypothetical production process

These blue, orange and red lines translate directly into the three link layers of the task network. The three layers M_1, M_2, and M_3 are constructed from this process structure as follows.

M_1: Machine-sharing (functional dependency). Tasks that use the same machine are connected. There is some functional dependency across these tasks, either because they need to share a resource, or because they require similar machine expertise. Functions 1, 2, and 3 are connected across products within the manufacturing functions. The fourth function, quality control (QC tasks: A-4, B-4, C-4, D-4), is connected across products as well. Tasks at different functions are not connected.

M_2: Product-sharing (workflow dependency). Each product follows a fixed production sequence or sequential workflow: Step 1 feeds into Step 2, which feeds into Step 3, and Step 4 (QC) inspects the final output. All pairs of tasks within the same product are therefore connected. No cross-product workflow dependency is assumed.

M_3: Goal conflict. Goal conflict arises when tasks have incompatible functional objectives. For each product, the manufacturing tasks (steps 1–3) prioritise throughput while the QC task prioritises defect detection — objectives that pull in opposite directions. This conflict is encoded as a tie between each product's manufacturing tasks and its QC task: A-1, A-2, and A-3 conflict with A-4; B-1, B-2, and B-3 with B-4; and so on. No conflict is assumed between QC tasks across products, nor between manufacturing tasks across products.

Visually, the three layers of the task-to-task network look as follows (Figure 18):

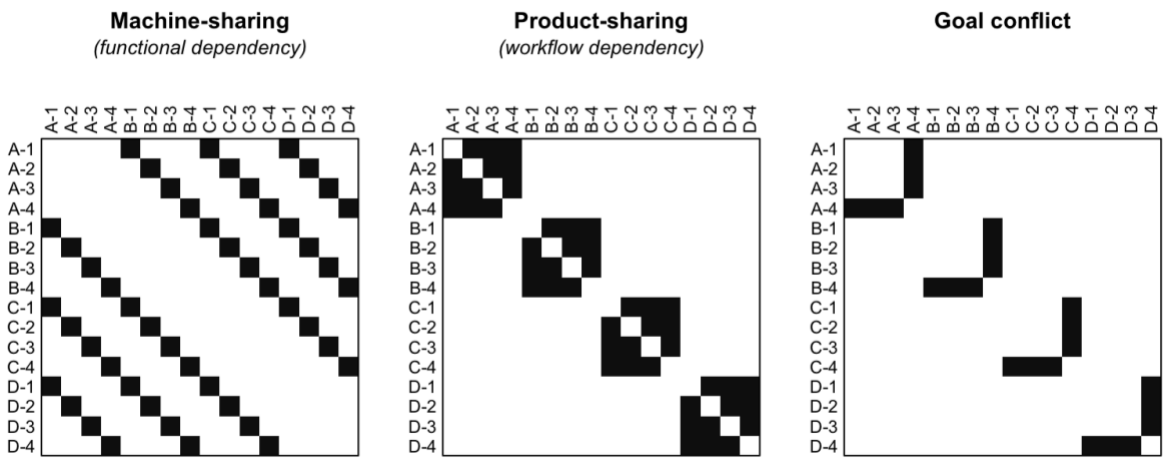


Figure 18 Simulated task matrices

Before applying the blockmodeling algorithm to real data in section 5, we test it on this simulated task network with a known structure. The purpose is to verify that the algorithm recovers the organisational forms one would theoretically expect from a given combination of dependencies and weights. The hypotheses are:

- **Weighting only M_1** (capturing machine-sharing or a functional dependency) should lead to a functional organisation (grouping tasks by machines).
- **Weighting only M_2** (capturing product-sharing or a workflow dependency) should lead to a product organisation (grouping tasks by products).
- **Weighting only M_3** (capturing goal conflict) should lead to a separate control department isolating control tasks.
- **Weighting all layers** should lead to hybrid arrangements combining elements of the above.

If the algorithm reliably produces expected structures from constructed input, we can apply it to empirical data with greater confidence in section 5.

4.3 Optimal partitions at different layer weights

Given the above 16 nodes and minimum cluster size of 3 nodes in the starting partition, the search for the optimal k will run from 1 to 5 clusters.

4.3.1 Putting all the weight on a single layer

Setting the weight of two layers to zero reduces the multiplex model to a standard single-layer blockmodel: the inactive layers contribute nothing to the criterion function and cannot influence the partition. Running the three models — one for each layer — confirms that the multiplex framework degrades gracefully to its single-layer special cases before any trade-off between layers is introduced and establishes the baseline positions from which each layer pulls the partition when given full weight.

When only M_1 receives weight (relWeights = 1, 0, 0), the algorithm groups tasks by machine requirements and recovers a **functional organisation** (Figure 19). The four clusters correspond to the four function types — steps 1, 2, and 3, plus quality control — each gathering tasks that perform the same function on the same machine regardless of which product they serve. A single distinct partition achieves the minimum criterion value.

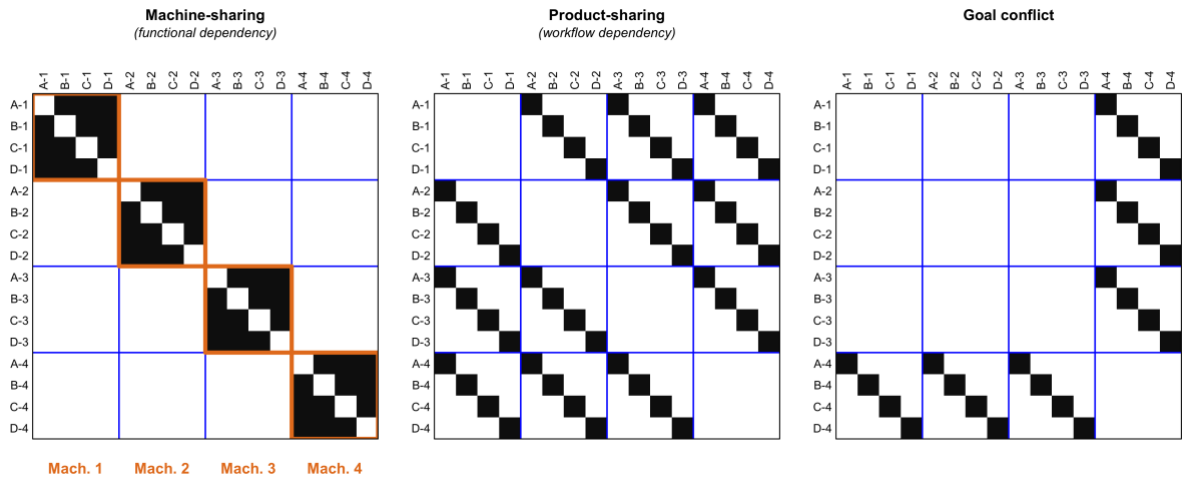


Figure 19 Optimal (functional) partition when weighting only machine-sharing ties

When only M_2 receives weight (relWeights = 0, 1, 0), the algorithm groups tasks by workflow dependency and recovers a **product organisation** (Figure 20). The four clusters correspond to the four product lines, each containing all four production steps for that product. Tasks are co-located because they depend directly on each other's outputs across successive steps. A single distinct partition achieves the minimum criterion value.

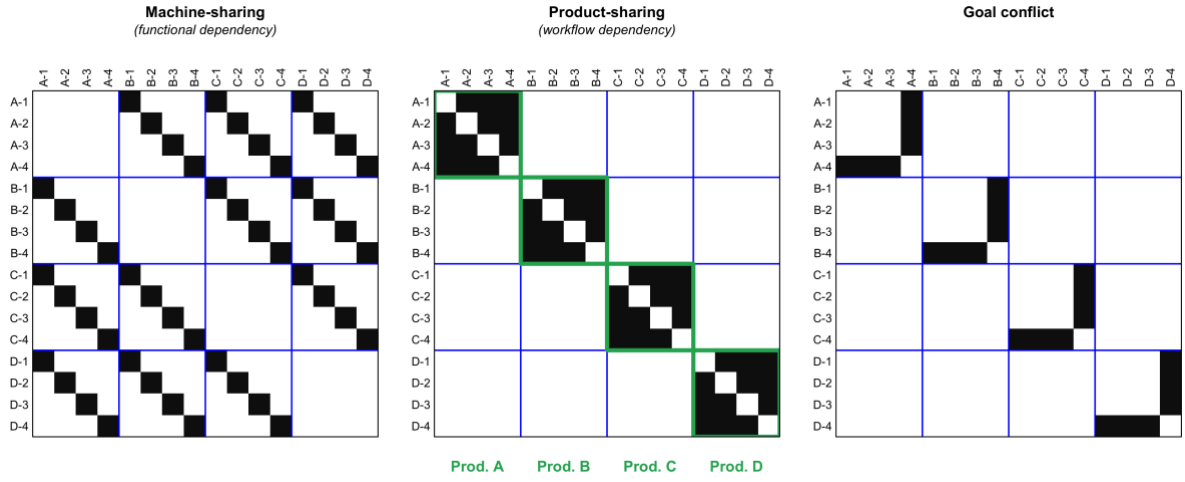


Figure 20 Optimal (product) partitioning when weighting only product-sharing ties

When only M_3 receives weight (relWeights = 0, 0, 1), the algorithm separates tasks with conflicting objectives and produces a two-cluster control structure at k = 2. All eight distinct partitions achieve zero criterion value. With four products and two clusters, the structural constraint (each product's QC task must be in the opposite cluster from that product's manufacturing tasks) generates exactly $2^4/2 = 8$ solutions and all 8 are found:

- **Seven optimal partitions are cross-product arrangements:** each cluster combines the manufacturing steps of some products with the QC tasks of others, distributing control responsibilities across the organisation rather than concentrating them. One of these 7 solutions is shown in Figure 21: cluster 1 takes the manufacturing steps of products A and B, but the control steps of C and D; cluster 2 manufactures products C and D, but controls A and B.
- **The eighth is the dedicated control department:** all four QC tasks in one cluster, all twelve manufacturing tasks in the other, represented in Figure 22.

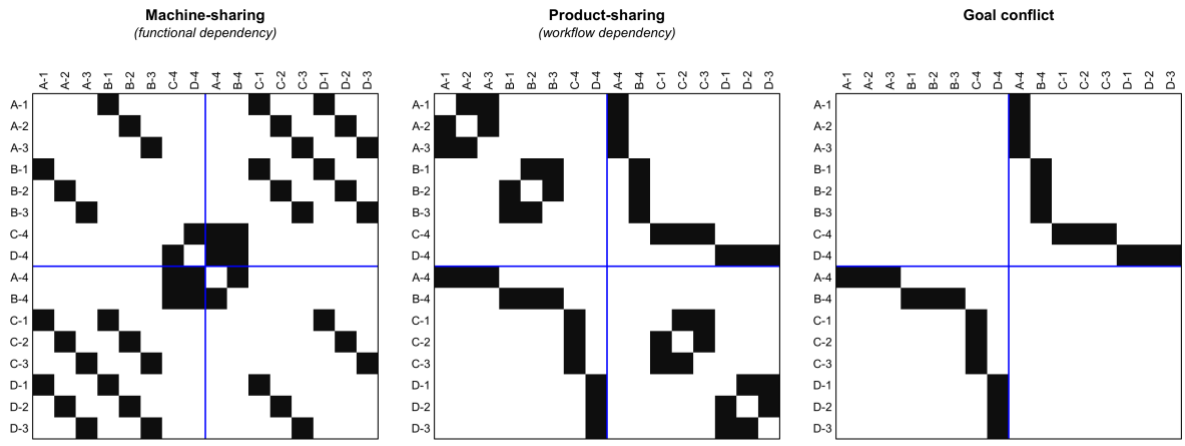


Figure 21 One of seven cross-product control structures, when weighting only conflict ties

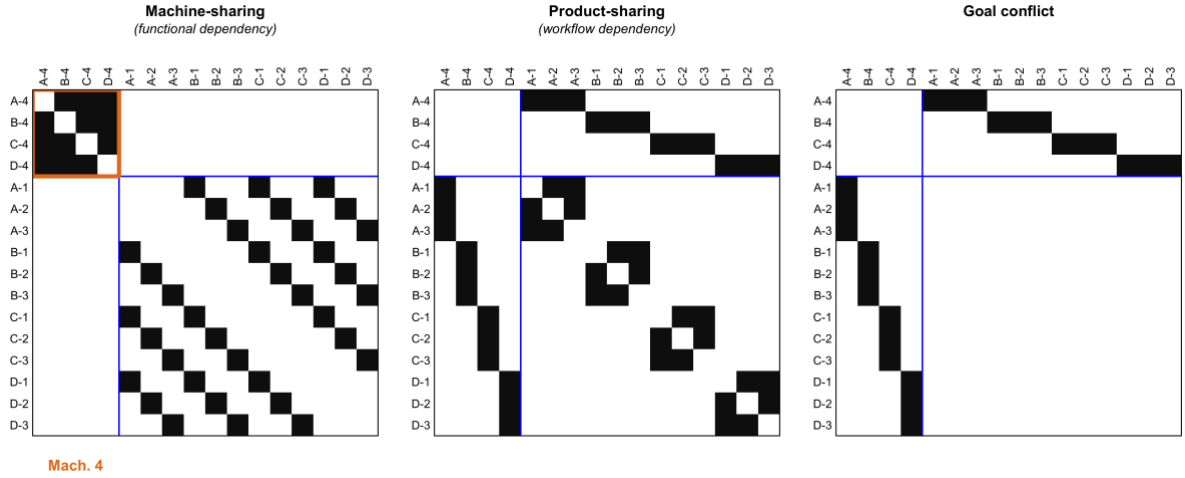


Figure 22 Dedicated quality control department when weighting only conflict ties

These three results are the reference points against which the multi-layer analyses in the sections that follow can be read: any shift in the optimal partition under combined weights reflects the cost of the design trade-off the multiplex model makes explicit.

4.3.2 Weighting all layers jointly

Three combined-weight scenarios are run, varying only the weight assigned to M_2. M_2 is the only layer pulling the design away from a functional (machine) grouping towards a workflow (product) grouping. It is thus sufficient to vary w_2 in order to discover different partitions. The equal-weight scenario ($w_1 = w_2 = w_3$) serves as the neutral baseline, where each layer receives the same standardised normal weight, namely 1 over the total number of pairs or $n(n-1)$. Next, two strategic scenarios test what happens when workflow coordination is assigned progressively larger weight in the criterion function.

Under equal weighting, the algorithm selects k = 4 and returns the **functional organisation** — the same partition as the single-layer M_1 result. The reason is that M_1 and M_3 pull in the same direction: both are satisfied at zero cost by a machine-based partition. Grouping all tasks of the same machine together satisfies M_1's complete-block requirement (tasks within each cluster share a machine) and M_3's null-diagonal requirement (the QC cluster is structurally isolated from the three manufacturing

clusters). M_2 thus bears the full residual — 96 raw violations — because a functional structure fragments every product workflow across four clusters. At k = 5 the total error is identical; parsimony selects k = 4. Under equal weighting, two of three rationales agree on the functional partition, and the coordination rationale cannot override them at equal cost.

Doubling M_2's weight (w_2 = 2 x nw) makes each workflow dependency violation twice as costly as a machine-sharing or conflict violation. The algorithm selects k = 4 and returns the **product organisation**: four clusters, each containing all steps of one product. M_2 error drops to zero; M_1 and M_3 are fully sacrificed at 96 and 24 raw violations respectively. This partition is identical to the single-layer M_2 result. A 2× coordination premium is sufficient to override both the machine-specialisation and goal-separation rationales entirely. The product organisation does not emerge from balanced weighting — it requires a deliberate commitment to coordination.

A workflow coordination premium of 1.4× (w_2 = 1.4 x nw) shifts the selected k from 4 to 5 and uncovers a third organisational form that does not appear in any single-layer analysis: four product-based manufacturing clusters plus one dedicated QC cluster, see Figure 23. The manufacturing clusters each contain the three production steps of one product; the QC cluster contains all four quality-control tasks. This structure satisfies M_3 at zero cost — the QC cluster has no within-cluster conflict ties — and distributes the remaining error across M_1 (60 raw violations: manufacturing clusters group tasks by product rather than by machine, mixing functional requirements) and M_2 (36 violations: the QC cluster separates QC tasks from the product workflows they depend on). Neither rationale is fully sacrificed.

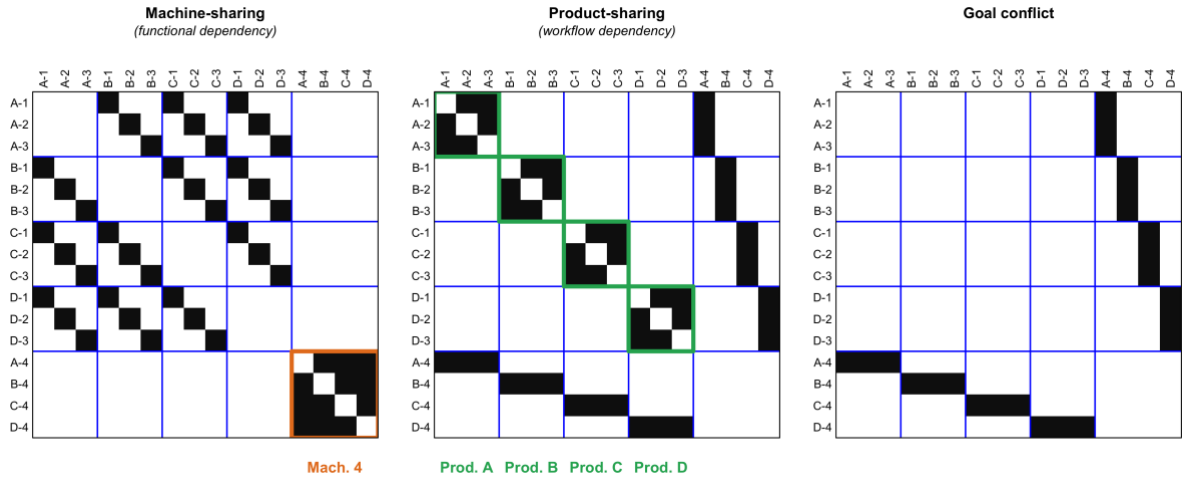


Figure 23 Product organisation with quality control department (weights = 1, 1.4, 1)

The 1.4× threshold lies below the level needed to produce the pure product organisation but above the level at which the functional structure remains optimal. The **product+QC form** is the joint optimisation's answer when workflow coordination matters more than under equal weighting but not enough to completely override conflict separation. This result is the most substantively relevant of the three: it is the only partition the algorithm finds that was not a single-layer optimum and that does not arise from any pre-specified ideal, emerging instead from the simultaneous trade-off among all three rationales.

Table 2 summarises the three scenarios. Each step up in coordination weight reduces M_2 error at the cost of M_1 and M_3 violations, until at 2× the partition converges on the pure product organisation. The intermediate 1.4x scenario is the only form that does not fully sacrifice any single rationale — and the only one whose structure was not pre-specified as a single-layer target.

Table 2 Three simulated scenarios with layer weights and resulting organisational form

Scenario	Weights	k	M_1 err	M_2 err	M_3 err	Organisational form
Equal	nw, nw, nw	4	0	96	0	Functional (machine)
2× M_2	nw, 2·nw, nw	4	96	0	24	Product (workflow)
1.4× M_2	nw, 1.4·nw, nw	5	60	36	0	Product + QC department

4.3.3 K-selection curve and sensitivity checks

Figure 24 shows the weighted criterion value $F(k)$ for $k = 1$ through 5 under each of the three combined-weight scenarios. All three curves decrease steeply from $k = 1$ to $k = 2$ and more gradually thereafter, reflecting that a single cluster fits the data poorly and that each additional cluster captures real structure up to a point.

The equal-weight scenario reaches its minimum at $k = 4$. The 2× M2 scenario also bottoms out at $k = 4$, with an identical value at $k = 5$; parsimony selects $k = 4$. The 1.4× M2 scenario is the only one that finds a lower value at $k = 5$ than at $k = 4$, which is why the algorithm selects five clusters rather than four. The small gap between $F(4)$ and $F(5)$ under the 1.4× scenario — and the absence of that gap under equal weighting — shows that the fifth cluster (the dedicated QC department) only pays off when workflow coordination is weighted high enough to make separating the QC tasks from their product teams worth the added M2 cost.

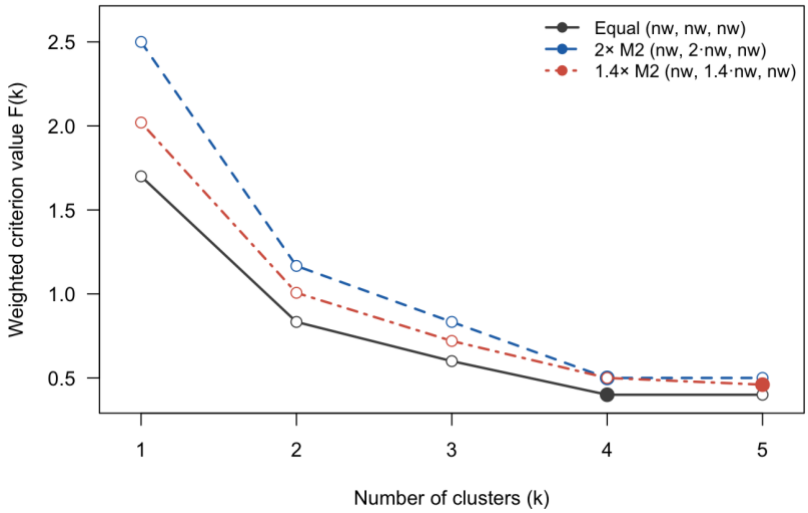


Figure 24 K-selection curve for simulated data

The results reported in this simulation study were obtained using a coarse 25 random restarts for the k-selection loop and more refined 200 restarts for enumerating all distinct optimal partitions (AOP) at the selected k. To assess whether the findings depend on these choices, the analysis was re-run across a grid of nine parameter combinations $REPS_KLOOP \in \{10, 25, 50\} \times REPS_AOP \in \{100, 200, 500\}$.

Across all nine configurations, every model returned the same optimal k, the same number of distinct partitions, and the same cluster compositions — including the lowest settings tested ($REPS_KLOOP = 10$, $REPS_AOP = 100$). Total render times ranged from 6.2 to 8.9 seconds (Table 3), with the AOP repetitions being the primary driver and k-loop having negligible effect across this range.

Table 3 Render time (seconds) across restart parameter combinations. Solutions are identical in all cells.

	AOP = 100	AOP = 200	AOP = 500
k-loop = 10	6.2s	7.0s	8.9s
k-loop = 25	6.3s	6.6s	8.4s
k-loop = 50	6.4s	6.7s	8.6s

4.3.4 Interactive online exploration tool

To support interactive exploration of the simulation study, the analysis described in this simulation stidy is implemented as a browser-based Shiny application, the *Multiplex Blockmodeling Explorer*, available at https://lmnu-blockmodeling.share.connect.posit.cloud. The code for this app is available in a public GitHub code repository: https://github.com/LMNU/blockmodeling

The application has a sidebar and a main results panel, see Figure 25.

The sidebar contains three groups of controls. The *first group* sets the layer weights w_1, w_2 and w_3 via sliders (range 0-3), adjusting the relative importance of functional dependency, workflow dependency, and goal conflict. The *second group* exposes the computation parameters (discussed in 4.3.3): the number of restarts for the k-selection loop, the number of restarts for finding all optimal partitions, the number of random seeds, and the minimum cluster size. The *third group* defines the simulated network itself: the number of products and the number of functions per product. Clicking *Run blockmodel* re-runs the analysis with the current settings.

The main panel then displays three outputs. A status banner reports the selected k^*, the weighted criterion value, and the parameter settings used. An error-by-k table shows the weighted criterion and per-layer errors for each k tested, with the selected k marked. A cluster assignment table lists the members of each cluster under the first distinct optimal partition. Below the tables, the three block matrices (one per layer) are redrawn with rows and columns reordered to the optimal partition and cluster boundaries marked.

The app is intended as a companion to the simulation study: it lets readers verify the results in section 4 directly, and to try alternative weight combinations or network configurations without modifying the underlying R code. It also allows to vary or expand the set of products and machines. Figure 25, for example, shows the result for products and three machines, finding an optimal solution of 6 product teams and 3 control teams – a total of 9 task clusters.

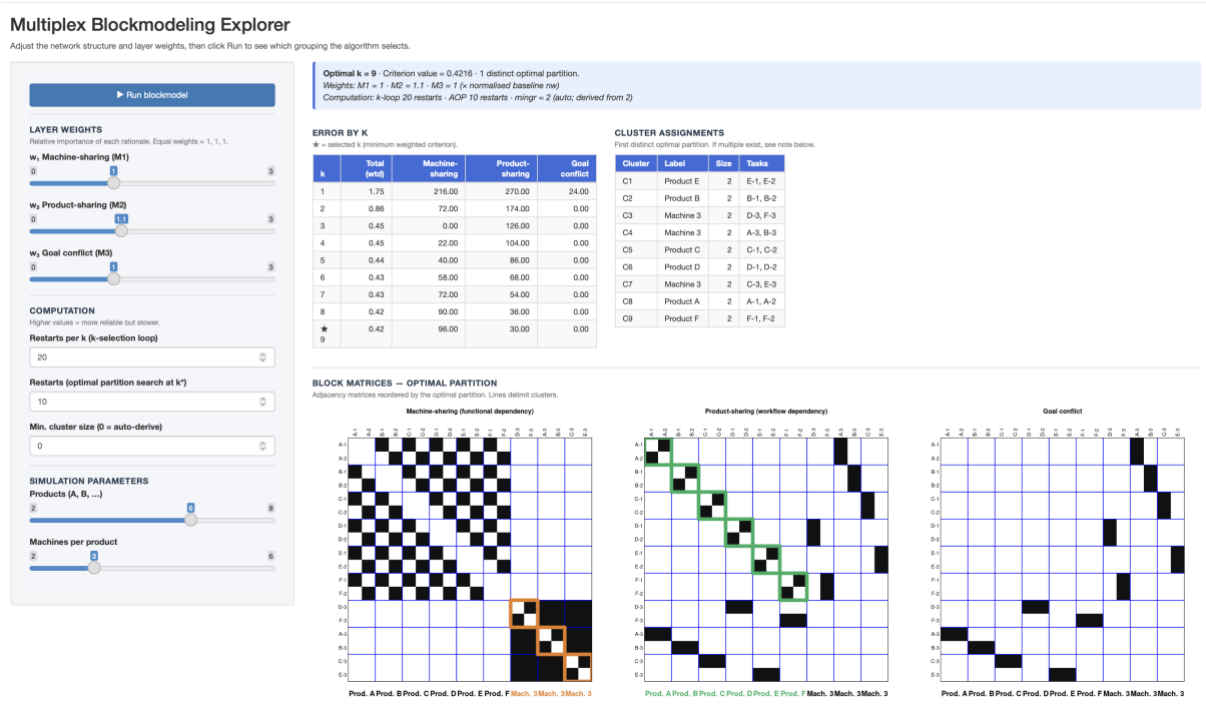


Figure 25 Screenshot of online interactive application “Multiplex Blockmodeling Explorer”

5 Results from empirical data set

Section 5 applies the blockmodeling framework to data from an FMCG factory in Belgium. Section 5.1 describes the production setting. Section 5.2 discusses the construction of the task network from the routing data. Section 5.3 reports the optimal partitions — first for a smaller subset of tasks used to calibrate the layer weighting and restart parameters, then for increasingly larger subsets of tasks.

5.1 Description of production setting and input data

Client's organisation question. The client is a factory in the Fast-Moving Consumer Goods (FMCG) industry in Belgium. They want to transform their organisation from a functional one (with departments organised around functionally similar machine groups) to one in which autonomous teams of workers can take end-to-end responsibility over a set of products and machines. They are thus looking for autonomous cells of cohesive product-machine partitions, with dense intra-cell linkages and sparse inter-cell links, in the tradition of Production Flow Analysis (see section 2.3.3). The data was obtained from a partner firm supporting the transformation and anonymised to protect the identity of both.

Raw input data (product x machine). The raw input data is a production flow or routing matrix: a binary product-by-machine matrix recording which products are processed on which machines. The production structure can be read directly from observed routing patterns. Figure 26 displays this routing matrix R showing which products are routed through which of the seven functions: Raw materials processing (R), Pre-production preparation (Pr), Cutting (Cu), Flavouring (F), Capping (Ca), Packing (Pa), Finishing (F). Some machines can combine multiple functions, like packing and finishing (Pa/Fi) or raw materials and cutting (R/Cu), creating a total of nine distinct machine groups.

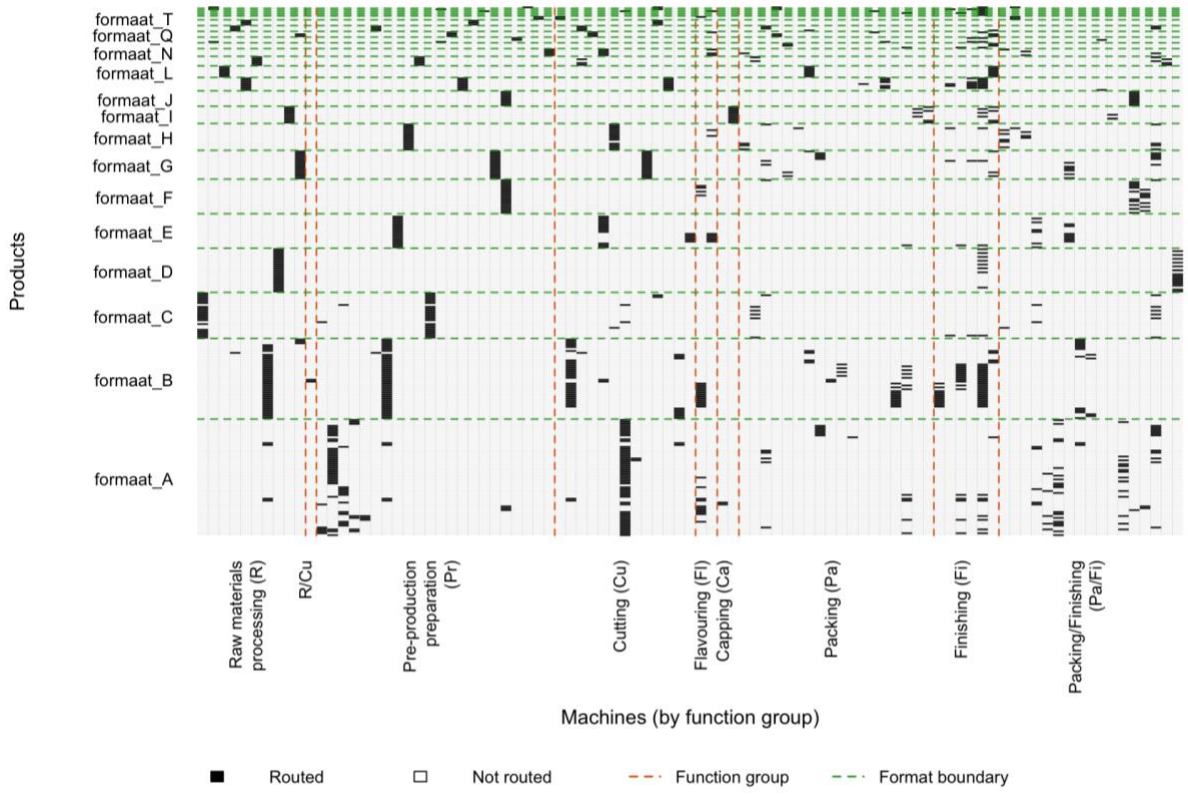


Figure 26 Raw input data (product x machine routing)

Descriptive statistics of the routing matrix. The full routing matrix contains 91 machines (across 9 machine groups) and 276 products (across 48 brands). The number of machines per functional group varies between 1 and 22, with a median of 10. The number of products per brand varies between 1 and 58, with a median of 3. A product typically requires processing by 4 machines, though this varies between 1 and 7. A machine typically handles 7 products, though this varies between 1 and 60.

Supplemented data from product catalogue. The routing data is supplemented with a product catalogue, listing product attributes such as brand, format and packaging size. There are 48 different brands, with the number of products per brand ranging from 1 to 58 and a median of 3. There are 25 different product formats (from small to large), with the number of products per format ranging from 1 to 61 and a median of 5. Finally, there are 15 different packaging sizes (from single-sizes to 100-pack sizes), with the number of products per pack size ranging from 1 to 96 and a median of 12.

Table 4 Descriptive statistics of the routing (product-machine) matrix

Category	Min	Median	Max
Products per brand	1	3	58
Products per format	1	5	61
Products per pack size	1	12	96
Machines per function group	1	10	22
Machines per product	1	4	7
Products per machine	1	7	60

5.2 Construction of the multilayer task-to-task network

The above-described product-to-machine routing matrix, combined with the product catalogue, is used to construct a symmetrical, multilayer task-to-task network as follows:

Node definition. The unit of analysis is a task: one machine operating on one product. A task $t = (p, m)$ exists wherever product p is routed through machine m, i.e. wherever $R_{pm} = 1$ in the routing matrix. Each non-zero entry in R defines one task, and it is these tasks — rather than products or machines individually — that form the nodes of the organisational task network. This definition aligns the analysis with the theoretical account in 3.2.2 and the simulation study in 4.2.

Layer definition. Six binary layers are constructed from the routing matrix R and the product attributes recorded in the product catalogue. All six are $T \times T$ symmetric matrices over the same set of tasks.

- *Machine sharing* (L1) links two tasks if they share a machine: $t = (p, m)$ and $t' = (p', m)$ are connected whenever the same machine m serves two different products. This layer captures resource contention and the grouping rationale of machine specialisation.
- *Product sharing* (L2) links two tasks if they share a product: $t = (p, m)$ and $t' = (p, m')$ are connected whenever the same product is processed on two different machines. This layer encodes workflow dependency: every step in a product's route depends on the steps before it.
- *Functional connections* (L3) links two tasks if their machines belong to the same functional group. Nine function groups cover all 91 machines. Where L1 captures direct connections between tasks on the same machine, L3 links tasks across different machines within a function, connecting tasks that share functional knowledge requirements without necessarily competing for the same resource.
- *Pack size connections* (L4) and *format connections* (L5) link tasks whose products share the respective attribute. Products with the same pack size follow similar packaging sequences;

products with the same format share physical dimensions that shape their machine requirements. Both layers represent product-attribute groupings that extend beyond direct workflow coupling.

- *Brand connections* (L6) links tasks whose products share a brand. Products of the same brand share design heritage and production know-how that cuts across format and pack size.
- The dataset contains no information from which goal conflict between specific task pairs can be derived. The analysis therefore includes no separating layer. This is flagged as a limitation in 6.2.

Final input data (task x task). Figure 27 shows the six task-network layers for the full empirical data with 1019 task-nodes, with orange dashed lines marking the boundaries of the company's current functional organisation — nine departments corresponding to the nine machine functions (seven unique functions and two groups combining functions).

- *L3 (Functional)* makes the logic of the current structure most visible: the large solid blocks along the diagonal confirm that current departments groups tasks that are densely functionally connected, while off-diagonal blocks are empty. The existing departments fits L3 perfectly by construction: the functional groups *are* the current partition, so every task is guaranteed to share a function with its cluster-mates and none with tasks outside.
- *L1 (Machine)* shows a finer-grained pattern within those same boundaries: each functional group contains several individual machines, visible as small diagonal clusters inside the larger functional blocks. As each machine belongs to a cluster, no machine-interdependencies are spread across the clusters.
- *L2 (Product)*, by contrast, shows ties scattered across functional boundaries, mostly in off-diagonal blocks. Product workflows cross many different functional groups, and these within-product, cross-machine dependencies are exactly what the functional structure leaves unmanaged. In their current functional organisation, the responsibility for a batch of products is handed over from department to department across the entire production flow.
- *L4–L6* reveal further structure across pack size, format, and brand, none of which aligns neatly with the functional grouping.

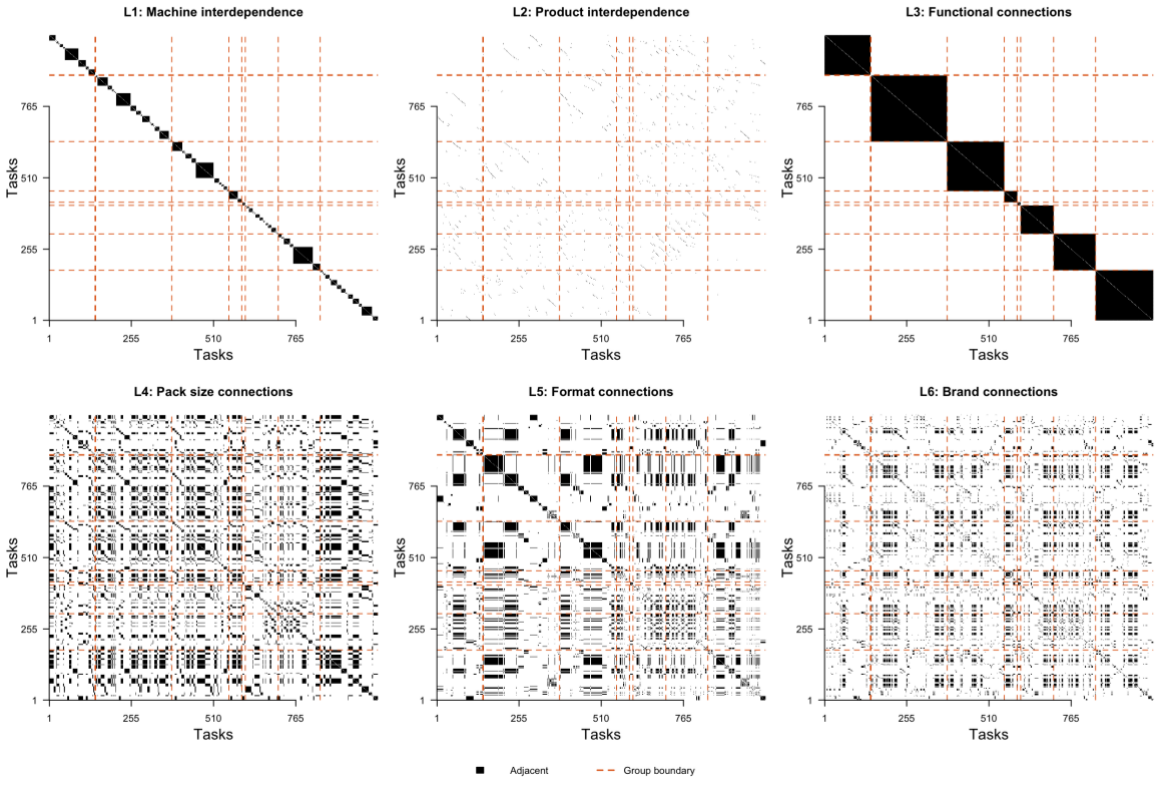


Figure 27 Current functional organisation (by machine grouping)

Descriptive statistics of the task-to-task network. Table 5 reports tie counts and densities across the six layers. The functional connections layer (L3) and pack size layer (L4) are the densest, with roughly one in six possible ties present, reflecting that most products share at least one functional step and that pack sizes repeat widely across the product range. Format connections (L5) and brand connections (L6) are moderately dense. Machine interdependence (L1) is sparse — most machine pairs are not directly connected through a product. Product interdependence (L2) is the sparsest layer by a wide margin, as most products are not linked by a shared machine at all. This variation in density across layers is what motivates density-equalised weighting: raw equal weights would effectively subordinate the sparse layers to the dense ones.

Table 5 Layer statistics: ties and densities

Layer	Ties	Density
L1 — machine interdependence	22400	0.0216
L2 — product interdependence	3374	0.0033
L3 — functional connections	167240	0.1612
L4 — pack size connections	166990	0.1610
L5 — format connections	128464	0.1238
L6 — brand connections	79904	0.0770

Adjustment to blockmodeling approach from simulation study. The core optimisation logic from the simulation study is retained, with two major changes.

Pre-specification: First, **no conflict layer** is included: all six layers use the same block type specification as M_1 and M_2 in the simulation, namely `comnul` — complete blocks on the diagonal, null blocks off-diagonal, and the algorithm partitions on grouping rationales alone.

K-selection: Second, **k is fixed at 3** rather than selected by minimising the criterion function across values of k, for both computational and organisational reasons. Computationally, the large number of tasks in the empirical setting compared to the simulation setting (1019 tasks vs 16 tasks) makes it impossible to search for the optimal number of clusters. But also organisationally, the client company had already decided to design three end-to-end product-owning teams – each of whom should own its own set of products and machines. The analysis therefore identifies the best partitioning of nodes (tasks, or product-machine actions) into three clusters given that constraint. Those identified clusters of tasks will be the responsibility or work terrain of teams. In what follows, the terms cluster and team are used interchangeably.

5.3 Optimal partitions for cohesive product-machine groupings

The full routing matrix contains 95 machines and 276 products. As discussed above, retaining all products produces a network of 1019 task nodes. This is too large for the optimisation to run at a useful restart count within reasonable time, especially while exploring different scenario's (different layer weightings). The analysis below therefore starts first with a selection of 2 brands that have 18 and 19 products respectively (37 products jointly) and jointly use 39 machines, leading to 139 task nodes (see section 5.3.1). This analysis is used to get a sense of the preferred specifications for the weighting of the layers and the necessary number of restarts for finding the global optimum. The final analysis includes all brands with at least 10 products for a total of 158 products and 73 machines leading to 561 tasks nodes (see section 5.3.2).

5.3.1 Optimal partition for a network of 139 tasks (37 products, 39 machines)

The analysis below runs the blockmodeling algorithm under seven weighting scenarios, varying which layers are active and how much weight each layer receives. The scenarios move from machine-rationale weights over product-rationale weights to product-machine cohesive strategies. This mirrors the strategic question of the client company: how much the new structure should be shaped by functional machine grouping, product attributes and product-machine linkages.

Scenario's 1 and 2: Functional partitioning. When only machine-related layers receive weight, either purely machine links with L_1 alone (w={1,0,0,0,0,0}) or adding the functional grouping of the machines, with L_1 and L_3 together (w={1,0,1,0,0,0}), the algorithm recovers a functional organisation in which tasks are clustered around machines. In both scenario's all 39 machines are assigned cohesively to a cluster. In scenario 1 the machine-level clustering disregards the functional grouping, but in scenario 2 the three clusters correspond to aggregates of the functional machines groupings. In both scenario's, no product is cohesive (processed fully in one cluster): all 37 products have their tasks distributed across clusters, so no team would have end-to-end ownership over any product's production steps. This is the structure the company currently operates and wants to move away from.

Scenario's 3 and 4: Product partitioning. When only the product-sharing layer receives weight in scenario 3 (w={0,1,0,0,0,0}), all 37 products become fully cohesive (meaning all tasks per product belong to the same cluster), but 26 of the 39 machines are split across clusters. Product-sharing alone separates products cleanly but does not produce a partition that uniquely allocates machines to clusters. Interestingly, adding the product-attribute layers in scenario 4 (pack size, format, and brand; w={0,1,0,1,1,1}) resolves most of the machine conflict: all 37 products remain fully cohesive and machine sharing drops to 7 of the 39 machines. The attribute layers thus provide a grouping signal that aligns product clusters with machine use even though they do not directly encode any machine relationships. This must mean that the product attributes determine to a certain extent which machines are used. Format is the main product attribute driving this, see further analysis below.

Scenario's 5, 6 and 7: Product-machine cohesive partitioning. The remaining three specifications explicitly weight both product-sharing and machine-sharing aiming to create internally cohesive product-machine clusters. Including product and machine links alone (scenario 5 with w={1,1,0,0,0,0}) gives a slight preference to machine-grouping (37 out of 39 machines cohesive) over product-grouping (31 out of 37 products cohesive). Adding all product-attribute layers alongside machine links (scenario 6 with w={1,1,0,1,1,1}) tips the balance again in the favour of machine groupings, producing the same cluster composition as the product-attribute specification above: 37 of 37 products and 32 of 39 machines cohesive. Finally, scenario 7 uses the same layers as scenario 6, but uses density-equalised weights, weighting each layer proportionally to its tie density. It finds a middle ground between scenario 5 and 6 with 34 of 37 products and 34 of 39 machines cohesive, and neither dimension fully sacrificed for the other.

Table 6 summarises all seven scenarios. The recommended partition, the density-equalised specification in scenario 7, is highlighted with the red box. Different product formats are produced on distinct sets of machines, so grouping tasks by product and format effectively also clusters them – to a large extent – by machine. As such, format is the strongest separator (within/between density ratio of 149) and brand barely separates at all (ratio of 2.3). Pack sizes are distributed more evenly across teams.

Table 6 Cohesion of products, machines, and attribute groupings across blockmodel scenarios.

Scenario	F	L1 err	L2 err	Products cohesive	Machines cohesive
FUNCTIONAL PARTITIONINGS					
1 - Machine links only (1,0,0,0,0,0)	5674	5674	6430	0 / 37	**39 / 39**
2 - All machine-related layers (1,0,1,0,0,0)	9272	5684	6548	0 / 37	**39 / 39**
PRODUCT PARTITIONINGS					
3 - Product links only (0,1,0,0,0,0)	5890	6566	5890	**37 / 37**	13 / 39
4 - all product-related layers (0,1,0,1,1,1)	17780	6050	6106	**37 / 37**	32 / 39
PRODUCT-MACHINE COHESIVE PARTITIONINGS					
5 - product & machine links (1,1,0,0,0,0)	11660	5682	5974	31 / 37	37 / 39
6 - all product layers + machine links (1,1,0,1,1,1)	23830	6050	6106	37 / 37	32 / 39
7 - density-equalised version of 6 (1/ties, l3=0)	26.63	5890	5934	**34 / 37**	**34 / 39**

Visual inspection of preferred partition. Figure 28 shows all six layers reordered by the preferred partition, with blue lines marking the three cluster boundaries, that define the task-boundaries of the teams.

- *The format layer (L5)* presents the clearest picture: three solid diagonal blocks, with virtually no off-diagonal ties, indicate that the three clusters or teams map almost perfectly onto distinct product format groups. Different product formats are produced on distinct sets of machines, so grouping tasks by product and format effectively also clusters them – to a large extent – by machine.
- *The product-sharing layer (L2)* shows similarly clean, though sparser, block-diagonal structure, with within-cluster ties concentrated along the diagonal and cross-cluster ties almost completely absent. This means the partitioning achieved its aim of each cluster or team handling the full set of tasks of their products.
- *The machine-sharing layer (L1)* is the most diagnostic for the machine-sharing question: the near-diagonal pattern within each cluster shows that most machines serve only one team, and the handful of off-diagonal dots between clusters correspond to the five machines that are shared across the teams.
- *The functional connections layer (L3)* shows within-cluster blocks but retains noticeable off-diagonal structure. This reflects that functional groupings cannot fully align with a product-based partition, as each product needs to pass through different functions to be fully processed.
- *The pack size layer (L4)* shows a noisier pattern, consistent with teams handling different pack sizes within their formats and brands.
- *The brand layer (L6)* is the weakest separator in this case: with two brands needing to be distributed across three clusters, the diagonal blocks are present but incomplete, and off-diagonal ties remain throughout. While Cluster 1 and 2 each comprise of mostly one brand, the third cluster is necessarily a mix of the two brands. This is an artifact of the selection of the subset of tasks, which was achieved by filtering out two brands from the full product catalogue.

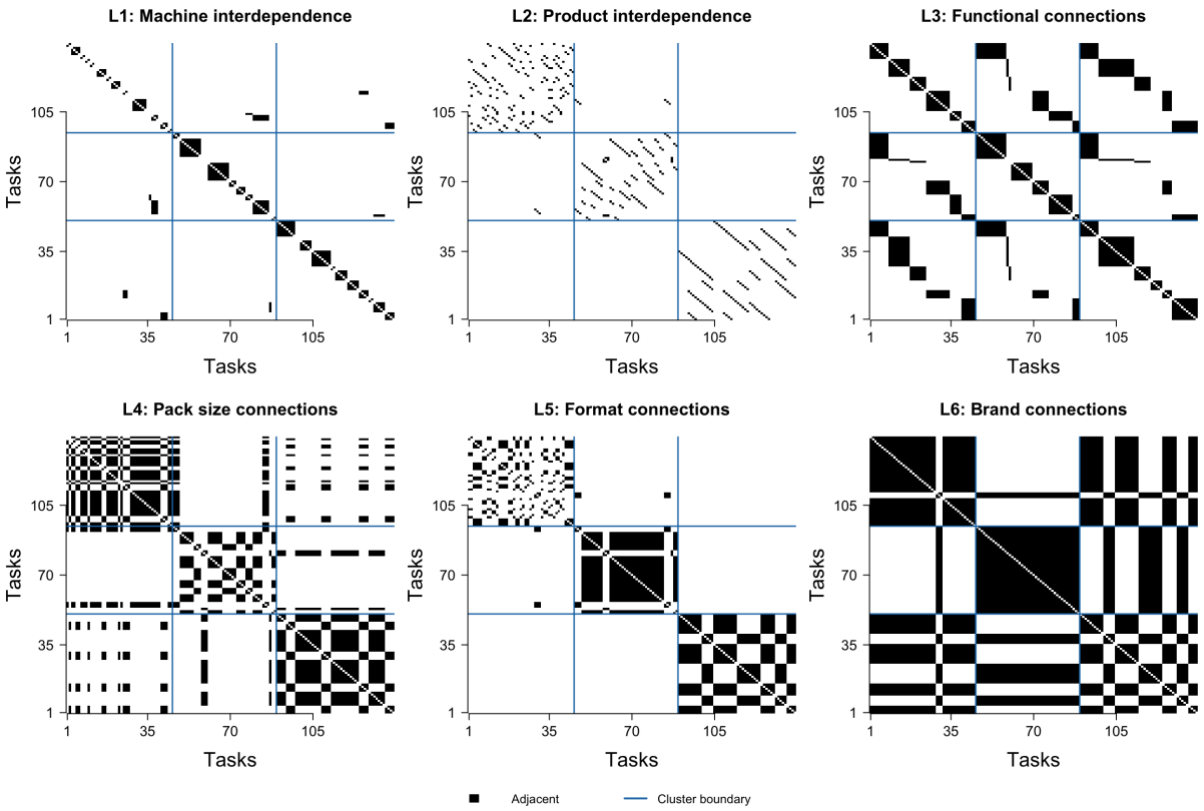


Figure 28 Optimal partition for product-machine cohesive teams

Block densities. The block densities of the six layers in Table 7 confirm the visual inspection above. The within-between density ratio is most informative, as it measures how concentrated ties are within clusters relative to between them — a ratio of 1 indicates no separation, and higher values indicate stronger alignment with the partition. *Format* (L5, ratio 149.1) is by far the strongest separator for the clusters: within-cluster density is 149 times larger than between-cluster. Both the *product* layer (L2, ratio 63.5) and the *machine* layer (L1, ratio 10.8) separate well, which is the aim for a product-machine cohesive clustering of tasks for end-to-end teams. The separation is not complete though – which would give a ratio of infinitive – indicating that some products and machines will be managed by multiple teams. The *pack size* layer (L4, ratio 5.9) separates more moderate, reflecting that pack sizes are repeated more evenly across clusters. The *brand* layer (L6, ratio 2.3) barely separates, consistent with two brands spread across three clusters. The functional L3 layer is not weighed in the criterion and thus does not separate at all (ratio 1.4).

Table 7 Block densities of the preferred partition

Layer	Overall	Within	Between	Within / between
L1 — machine	0.033	0.084	0.008	**10.8**
L2 — product	0.021	0.064	0.001	**63.5**
L3 — functional	0.142	0.178	0.124	1.4
L4 — pack size	0.220	0.491	0.084	5.9
L5 — format	0.151	0.452	0.003	**149.1**
L6 — brand	0.497	0.794	0.351	2.3

Cluster composition. Table 8 gives more insight into the cluster composition of the preferred partition. Clusters 1, 2 and 3 manage 9, 10 and 15 unique products respectively. The remaining three products that are spread across clusters are shared only between cluster 1 and 2, while cluster 3 doesn't share any product with another team. In practice, this means that team 1 and 2 will need to organise a product handover when the product moves between the teams.

The five machines that are spread across clusters are shared between all three clusters. In practice, this means that the management of those five machines will be shared between three teams. The teams will need a shared planning system to schedule their products across these five machines.

Finally, all-but-one formats are uniquely attributed to the teams, allowing the teams to specialise their expertise into specific formats and also allows for easy allocation of potential new products to the teams, based on format. The one shared format, is format L and is handled both by team 1 and 2.

Table 8 Cluster composition of preferred partition (density-equalised weights).

Cluster	Tasks	Unique products	Shared product	Unique Machines	Shared machines	Formats
1	45	9	3	16	4	B, H, L, M, R
2	44	10	3	9	3	G, I, L
3	50	15	0	9	3	C, E
Total		34/37	3/37	34/39	5/39	

Sensitivity checks on the number of repetitions. To calibrate the number of random restarts, the algorithm was run at 50, 100, and 200 repetitions across all seven scenarios. Six of the seven produce identical criterion values, cluster compositions, and cohesion counts at all three settings — including the density-equalised specification that is carried forward as the preferred partition. The one scenario that does not converge at 50 reps is the sparse product-and-machine-links specification (w={1,1,0,0,0,0}), which finds a suboptimal solution at 50 reps (F = 11,680) before settling at F = 11,660 at 100 and 200 reps. Since this specification is not carried forward, the instability has no bearing on the main results. Runtime scales roughly linearly with repetitions: 50 reps take just over two minutes, 100 reps under four, and 200 reps just under eight. Table 9 summarises the sensitivity checks.

Table 9 Render times by number of repetitions (7 scenarios, 37 products, 39 machines, 139 task nodes)

REPS	Total render time	Scenarios with identical solutions to 200 reps
50	2 min 4 sec	6/7
100	3 min 53 sec	7/7
200	7 min 47 sec	7/7

Moving to the analyses on the larger task networks in the next section, 50 repetitions will be used on the preferred scenario – the density equalised weighting of the machine layer and all product-related layers.

5.3.2 Scaling the analysis to larger task networks

The 139-node task network is small enough to try out different weighting specifications and numbers of random restarts. It shows that scenario 7 consistently outperforms the others when the goal is to create product-machine cohesive groupings of tasks, and the sensitivity analysis confirms that 50 restarts produce reliable convergence for that (and all-but-one) specification(s). The analysis on the larger task networks is therefore restricted to only the density-equalised scenario 7 — weights inversely proportional to layer density, with L3 excluded — and only 50 repetitions.

The task network is expanded by progressively relaxing the brand filter. Retaining brands with at least 17 products expands the network from two to three brands: the two brands above (that jointly comprise 37 products) plus the largest brand overall (with 58 products), for a total of 95 products across 58 machines, creating 372 tasks. Retaining brands with at least 10 products brings in five additional brands, for a total of eight brands, or 158 products across 73 machines, creating 561 tasks.

The number of clusters is kept at 3 and the minimum group size for the random starting partitions is set at one sixth of the number of tasks (so a minimum group size of 62 for the 372-node network and 93 for the 561 node-network). Nonetheless, both blockmodels produce a three-cluster solution with near-equal cluster sizes (see Table 10 for a side-by-side comparison). In the 372-task network, the three clusters hold 126, 123, and 123 tasks, grouping 30, 33, and 32 products respectively. In the 561-task network they hold 191, 183, and 187 tasks across 57, 50, and 51 products. As in the analysis of the smaller task network, it appears that the product, machine and product-attribute layers provide enough differentiating signal to pull tasks into three comparably-sized groups.

The optimal partitions for both network sizes are shown in Figure 29 and Figure 30.

Product cohesion is near-perfect in both cases: all 95 products land cohesively in the 372-task network; 156 of 157 do in the 561-task network. This is a stronger result than in the 139-task analysis, where only 34 of 37 products landed cohesively under density-equalised weights.

Machine cohesion moves in the opposite direction. In absolute terms it rises (34 of 39 machines at 139 tasks, 44 of 58 at 372 tasks, 54 of 73 at 561 tasks), but proportionally it falls from 87% to 76% to 74%. As more brands enter the analysis, machines need to process more and more products. Machines serving products from multiple product-attribute groups thus become more common, and the density-equalised specification resolves the conflict in favour of product ownership and product-attribute cohesion. The result is that while near-complete product cohesion is achievable across all network sizes, the share of machines that cross cluster boundaries grows as the scope expands, requiring more explicit management of shared machine capacity.

Format remains the sharpest separator: in the 372-task network, the within-cluster to between-cluster density ratio on L5 reaches 57.0, against 6.2 on the machine layer and infinite on the product layer. In the 561-task network, the within-between ration on the format layer increases again to 151 and the product layer achieves a ratio of 914.2, while the machine layers stays at 8. In both network sizes, product-sharing ties are almost entirely contained within clusters, largely thanks to a format-based partitioning.

Brand cohesion, largely absent in the two-brand analysis, emerges incrementally. As more brands get added, more opportunities arise for splitting them across clusters. At 372 tasks, one of three brands is cohesive, while this increases to three of eight at 561 tasks. Brands with more varied product portfolios — spanning multiple formats or pack sizes — are the ones that split.

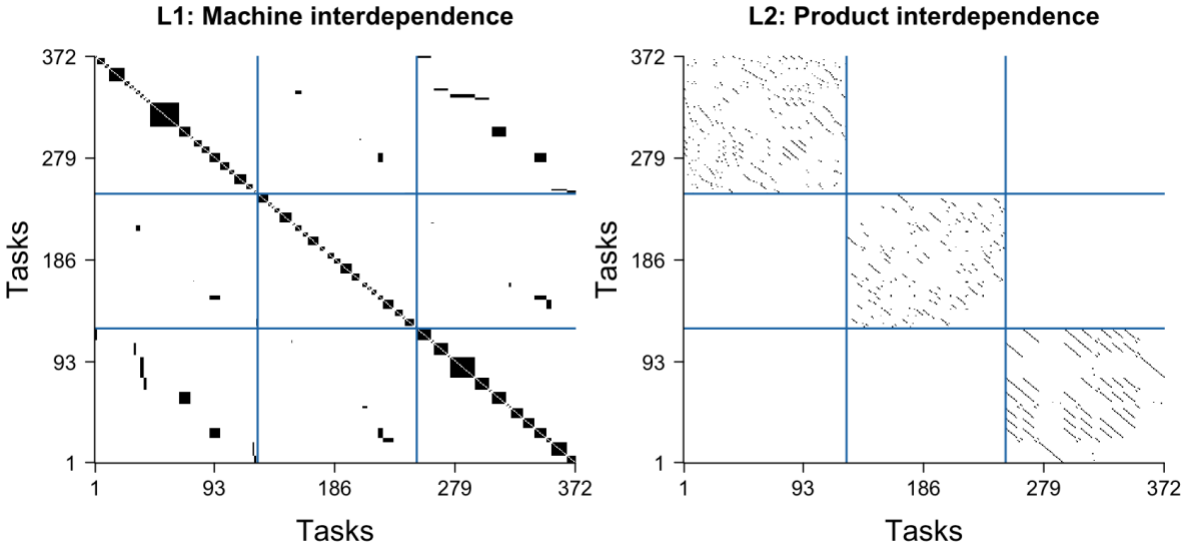


Figure 29 Optimal partition of 372-node task network shown on two layers

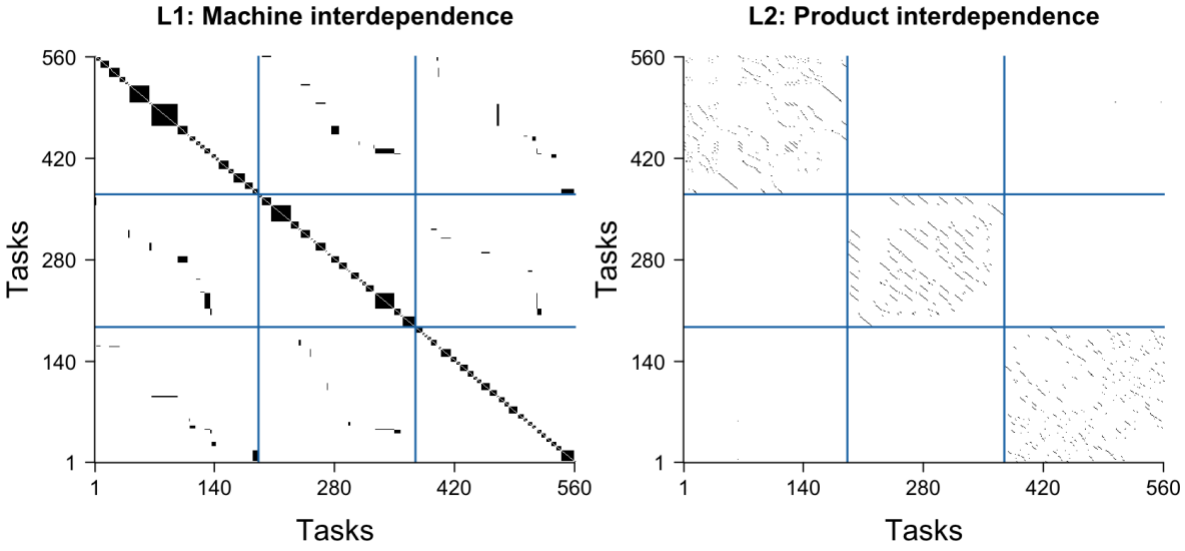


Figure 30 Optimal partition of 561-node task network shown on two layers

To conclude, product-machine cohesive clusters can be found to a certain extent across all three network sizes, with shifting trade-offs for machine cohesion. As more brands and products are introduced, product ownership becomes near-complete within clusters. The cost is in machine sharing: adding more brands requires machines to serve multiple product groups, and the preferred specification resolves this by prioritising product (attribute) cohesion over machine one. For the company, this means that more product variety it wishes to offer, the more machines will need to be shared between the product teams.

Computational cost scales sharply and non-linearly with network size. At 50 restarts, the 139-task analysis completes in just over 2 minutes; the 372-task analysis takes 37 minutes; the 561-task analysis runs for about 6 hours. The jump from 372 to 561 tasks is disproportionately large — roughly a sixteen-fold increase in runtime for a 1.5-fold increase in tasks — reflecting both the quadratic growth of the T×T adjacency matrices and the harder optimisation landscape when more brands introduce more cross-brand variation for the algorithm to resolve.

Table 10 Comparison of blockmodel on two larger network sizes

	139 tasks	372 tasks	561 tasks
Network expansion			
Brand filter	Brands with 17 to 20 products	Brands with ≥17 products	Brands with ≥10 products
Network overview			
Products (P)	37	95	158
Machines (M)	39	58	73
Tasks (T)	139	372	561
Brands	2	3	8
Optimal partition — density-equalised weights (L1, L2, L4, L5, L6)			
Number of clusters	3	3	3
Tasks per cluster	45 / 44 / 50	126 / 123 / 123	191 / 183 / 187
Products per cluster	12 / 13 / 15	30 / 33 / 32	57 / 50 / 51
Machines per cluster	20 / 12 / 12	25 / 32 / 16	26 / 31 / 37
Weighted criterion (F)	26.63	49.29	77.21
Cohesion results			
Products cohesive	34 / 37	95 / 95	156 / 157
Machines cohesive	34 / 39	44 / 58	54 / 73
Brands cohesive	0 / 2	1 / 3	3 / 8
Formats cohesive	8 / 9	11 / 13	15 / 17
Pack sizes cohesive	2 / 5	4 / 8	5 / 10
Function groups cohesive	2 / 8	0 / 8	0 / 8
Block density separation (within / between)			
L1 — machine	0.084 / 0.008 = 10.8×	0.060 / 0.010 = 6.2×	0.053 / 0.007 = 8.0×
L2 — product	**0.064 / 0.001 = 63.5×**	**0.029 / 0.000 = ∞**	**0.017 / 0.000 = 914×**
L3 — functional	0.178 / 0.124 = 1.4×	0.165 / 0.145 = 1.1×	0.190 / 0.141 = 1.4×
L4 — pack size	0.491 / 0.084 = 5.9×	0.269 / 0.202 = 1.3×	0.280 / 0.125 = 2.2×
L5 — format	**0.452 / 0.003 = 149×**	**0.446 / 0.008 = 57.0×**	**0.347 / 0.002 = 151×**
L6 — brand	0.794 / 0.351 = 2.3×	0.753 / 0.313 = 2.4×	0.373 / 0.154 = 2.4×
Runtime (50 restarts)			
Total render time	2 min 4 sec	36 min 52 sec	≈ 6 hours

6 Discussion

This chapter reflects on the analysis as a whole. Section 6.1 draws out the broader implications of the framework for organisational design practice. Section 6.2 identifies the main limitations — both methodological, ontological and epistemological. Section 6.3 points to directions for future research.

6.1 Implications for organisation design practice

The most direct implication of the framework is that **data already collected for operational purposes** can be repurposed for designing organisational structures. Routing matrices exist in most manufacturing firms as a by-product of production planning, but similar ones may exist in other types of organisations given the widespread and increasing digitalisation of work processes, planning and control systems. The current analysis derives its input entirely from such a matrix, with no surveys, interviews, or other process documentation. Organisation design analysis that previously required manual data gathering through workshops or ethnographic fieldwork can now be run from the production system.

Because the **analysis can be re-run** whenever the routing matrix is updated — as products are added, discontinued, or rerouted — firms can treat structural fit as something to **monitor continuously** rather than a question that is settled once and revisited only during major reorganisations. A structure that is optimal for today's product mix may no longer be optimal three years later. The framework makes that drift visible. It may also signal when increasing levels of cross-unit interdependencies are putting large coordination strains on organisations and flag when a re-fit is necessary.

The method is feasible with **limited computational resources**. Running more than 50 random restarts quickly becomes infeasible for a realistic number of products and machines, as runtime increases in a cubic-like way with the number of tasks. However, the sensitivity analyses in this thesis have shown that 50 random restarts can be sufficient, if enough layers in the task network contain signals that drive the algorithm towards the optimal partition for the chosen strategic direction. Namely, for a desired product-machine cohesive partitioning, 50 random restarts were not sufficient to find the global optimum when only product and machine interdependencies were considered. However, when additionally adding the product attribute layer ‘format’ – which drives much of the machine selection – the global optimum was found with 50 repetitions.

The weighting system **operationalises contingency theory** in a concrete and transparent way. The claim that optimal structure depends on the organisation's context and priorities is well-established in the literature, but applying it has historically relied on managerial judgment. The current framework forces that judgment to become explicit: before the algorithm runs, the organisation must assign weights to competing rationales for grouping and separating activities. Deciding that workflow coordination receives 1.4 times the weight of machine-sharing is a strategic position, and making it numerical makes it discussable, revisable, and comparable across scenarios.

Finally, the framework also produces outputs that are **fully reproducible and auditable**. The partition is derived from observable data through a transparent optimisation criterion. In the context of labour relations and change management, the method allows management and worker representatives to scrutinize a restructuring based on production data rather than on individual discretion. The reorganisation is easier to explain and contest on substantive grounds.

6.2 Limitations of the current analysis

The current analysis faces several limitations that affect how its results should be interpreted.

6.2.1 Methodological and organisational limitations

The most consequential limitation of the empirical application is the **absence of a goal conflict layer**. The theoretical framework in section 3 and the simulation study in section 4 specify three types of task dependency: machine-sharing (or resource-sharing more in general), product-sharing (or workflow dependency), and goal conflict. However, the empirical routing data contains no information from which goal conflict between task pairs can be derived. The current analysis therefore partitions tasks on grouping rationales alone, without the separating logic that would push conflicting tasks into different clusters.

A further limitation of the current analysis is the use of **binary ties**. Using presence-absence edges instead of valued ties simplifies the optimisation but discards information about how frequently or intensively two tasks interact. In a production setting, not all machine-sharing ties are equally consequential: two tasks that share a machine for one product run per week impose a different coordination burden than two tasks sharing a machine that runs continuously. The current analysis treats these as equivalent.

The **number of clusters was fixed** at $k = 3$ by the client's organisational decision rather than derived from the data. The criterion function is therefore never minimised freely, and the analysis cannot determine whether three teams is the number the data would recommend in the absence of that constraint. It is possible that other numbers of clusters would produce a structurally cleaner partition. Searching over a set of K different k's would increase the run time linearly by K.

The current partition also does **not balance workload** across the three teams. Though the clusters are not too differently sized — both in the number of tasks assigned to each team and in the number of products and machines they cover — the monthly production volumes (or variation thereof) have not yet been considered. This means that the transition to end-to-end teams would likely require additional decisions about staffing and capacity allocation that the current blockmodeling framework does not address.

Finally, a limitation shared by any task-network approach is that it treats tasks as the unit of analysis and says nothing about the **people** who perform them. Skills, experience, informal relationships, and tacit knowledge are absent from the routing matrix. A partition that looks clean in the task network may be difficult to implement if the required human expertise to execute those tasks is not distributed accordingly.

6.2.2 Ontological and epistemological critiques

The framework rests on an objectivist ontology: the task network is treated as a representation of a real, pre-existing production structure that can be mapped and partitioned to design a social structure of teams and departments. Several critiques follow from that stance.

The **first** is that **the production structure is itself socially constructed** (Bijker, 1987). The routing matrix records which tasks exist, which machines handle them, and in what combinations, but each of these reflects decisions made by people over time: which products to develop, whether to automate production or retain artisanal methods, how to decompose a process into discrete tasks, which machines to invest in. These are not natural features of the work; they are accumulated outcomes of

management choices, market pressures, and historical contingencies. The task network is therefore not a fixed substrate waiting to be analysed, but an artefact that already encodes prior strategic, organisational and technological decisions.

The practical implication is that the framework is most defensible when the current task structure is stable and likely to persist, even when a new organisational structure is desired for strategic reasons. Manufacturing is a relatively favourable setting: the physical logic of production – while perhaps shaped by earlier management choices – does create relatively stable and persisting task dependencies, regardless of which people, teams or units handle them. The critique bites harder in project-based or knowledge-intensive organisations. In a consultancy or research group, the projects a team works on are often shaped by the team's existing expertise, preference, network, and reputation, so the project portfolio partly *reflects* the current structure rather than *preceding* it. Applying blockmodeling to the project-team assignment matrix in such a setting could reproduce the existing structure rather than identify an alternative one, because the input data is co-constituted with the structure it is meant to inform.

A **second** critique concerns the **epistemological assumptions of rational optimisation in design**. Minimising a criterion function derived from a routing matrix looks, at first glance, like a continuation of scientific management: decompose work into measurable tasks, apply formal methods, find the optimal arrangement. The charge would be that this is an extreme form of Taylorism.

The charge however misidentifies what is being optimised. Taylor's project was to rationalise the *execution* of individual tasks: through time-and-motion studies, to find the one best way to perform each physical movement and prescribe it to workers as a binding method. The objective was to remove discretion from the shop floor and transfer it to management. The framework developed here addresses an entirely different level of the design problem. It does not optimise *how* tasks are performed; it identifies *which* tasks should be grouped into the same team. Once that grouping is established, end-to-end teams are largely self-governing within their product domain: sequencing their own work, managing their own bottlenecks, coordinating internally without reference to a central authority. The logic is closer to the sociotechnical design tradition (de Sitter et al., 1994), which explicitly sought to create self-regulating units with sufficient internal variety to manage their own task environments, than to Taylorist decomposition. Where Taylor separated conception from execution tasks, this framework is designed to restore their integration within product-and-machine-owning teams.

A **third** more substantive epistemological concern is **whether the criterion function adequately represents what matters for organisational design**. Reducing three distinct organisational values — specialisation of resources, coordination of workflow interdependence, and separation of conflicting goals — to a single weighted score presupposes that these values are commensurable: that the organisation can and should trade off a reduction in machine-sharing against an increase in product cohesion on a common scale. There are design decisions — separating quality control from production, for instance — that may be driven by compliance requirements, professional ethics, or power considerations that cannot be fully expressed as a weight in an optimisation. The framework is best understood as a tool for outlining the design space and making trade-offs explicit — useful both as an analytical instrument in practice and as a pedagogical device for teaching organisation designers to reason about competing structural rationales — not as a substitute for organisational judgment about which trade-offs are acceptable.

6.3 Future research

A natural next step would be to test whether the organisational structures identified by the framework as optimal translate into **measurable organisational outcomes**. The simulation and empirical

analyses evaluate candidate structures against task-derived criteria, but leave open whether the most cohesive partitions produce better performance at the firm level. Linking blockmodel-derived structural recommendations to outcome data — productivity, throughput time, quality metrics — would require longitudinal designs that track organisations before and after structural interventions, or cross-sectional comparisons across organisations (or departments) with varying degrees of fit between their actual structure and the framework's recommendations. This is beyond the scope of the current thesis, but it is the most direct route to validating the prescriptive claims the framework makes.

A related extension is the development of **data-driven methods for selecting layer weights**. The current framework requires weights to be specified in advance, which grounds the analysis in organisational strategy but also means the output is sensitive to choices that may be difficult to justify precisely. Methods that endogenously set the layer weights – perhaps through the outcome analysis described in the previous paragraph – would reduce reliance on a priori specification.

Finally, **social communication data** offers a different angle. If interaction logs, email data, or other coordination records are available alongside the task routing matrix, they can be used to map the organisation's informal structure — who actually coordinates with whom. Comparing that informal network against both the formal structure and the task network would reveal whether coordination in practice aligns more closely with the underlying task dependencies or with the official organisation chart. Where the informal structure tracks the task network but diverges from the formal one, the framework's recommendations may find empirical support in the organisation's own behaviour.

7 Conclusion

Designing an organisational structure is **analytically hard**. Organisations consist of hundreds of tasks connected through multiple types of dependency simultaneously — some tasks share knowledge or resource requirements, others depend workflow-wise on each other's outputs, others conflict in their functional goals. Grouping tasks to manage one type of dependency typically worsens another. The theoretical literature has documented these trade-offs in detail, but has not translated them into tools that help practitioners make concrete structural decisions. The data typically available to study organisational structure are too coarse to inform unit design at the level of tasks and activities. And the relational data that network analysis has developed to study organisations describes who *currently* coordinates with whom, rather than who *should* coordinate with whom, given their task dependencies.

This thesis proposes a different starting point. Rather than mapping the *social structure* that has formed around an existing design, it maps the **task structure** that should inform a new one. Several disciplines — socio-technical systems analysis, engineering design, factory layout and organisation science — have (largely) independently converged on matrix representations and partitionings of task interdependencies, arriving at the same formal logic through entirely separate intellectual traditions. Blockmodeling provides the analytical machinery to turn those representations into a structural recommendation.

The framework developed here **integrates three grouping rationales** — specialisation of resources, coordination of workflow interdependence, and separation of conflicting goals — into a single **multiplex blockmodeling criterion**. Each rationale corresponds to a distinct relational layer and each layer receives a weight that reflects the organisation's strategic priorities. The criterion function then finds the partition of tasks that minimises total inconsistency across all layers simultaneously. Where earlier approaches applied blockmodeling for organisation analysis in a diagnostic mode — uncovering structure from observed communication data — this thesis applies it prescriptively to the task network, identifying what the optimal organisational structure should be rather than what the current one is.

A **simulation study** confirms that the algorithm recovers the structures theory predicts: a functional organisation when prioritising machine-dependencies, a product organisation under product-dependency weights, and a hybrid product-plus-quality-control structure under intermediate weights that does not appear in any single-layer analysis. The **empirical application** to a FMCG manufacturer shows that the framework works on real production data. Weighting the machine-related layers reproduces the company's currently existing functional organisation, under which every product's tasks are split across machine clusters and products must be handed over from one functional group to the next. As an alternative to this functional structure, relatively internally dense, externally sparse machine-product cohesive groupings of tasks can be found when weighing both machine-dependencies, workflow-dependencies and product attribute links like brand, format, and pack size.

The input data required for this analysis — a routing matrix recording which products pass through which machines — exists in most manufacturing firms as a by-product of production planning. No surveys, interviews, or process documentation are needed. The framework offers practitioners a replicable, auditable way to explore structural options: organisations can examine what their production data implies under different strategic priorities before implementing any changes.

8 References

Aghion, P., Bloom, N., Lucking, B., Sadun, R., & Van Reenen, J. (2021). Turbulence, Firm Decentralization, and Growth in Bad Times. *American Economic Journal: Applied Economics*, *13*(1), 133–169. https://doi.org/10.1257/app.20180752

Albert, D., Eklund, J. C., & Tang, L. (2025). A new organizational structure database: Examining structure through top management team compositions. *Strategic Management Journal*, *n/a*(n/a). https://doi.org/10.1002/smj.70029

Ashby, W. R. (1958). Requisite Variety and Its Implications for the Control of Complex Systems. *Cybernetica*, *1*(2), 83–99. https://doi.org/10.1007/978-1-4899-0718-9_28

Baldwin, C. Y., & Clark, K. B. (2000). *Design Rules, Volume 1: The Power of Modularity*. The MIT Press. https://doi.org/10.7551/mitpress/2366.001.0001

Barney, J. (1991). Firm Resources and Sustained Competitive Advantage. *Journal of Management*, *17*(1), 99–120. https://doi.org/10.1177/014920639101700108

Barrick, M. R., Thurgood, G. R., Smith, T. A., & Courtright, S. H. (2015). Collective Organizational Engagement: Linking Motivational Antecedents, Strategic Implementation, and Firm Performance. *Academy of Management Journal*, *58*(1), 111–135. https://doi.org/10.5465/amj.2013.0227

Batagelj, V. (1997). Notes on blockmodeling. *Social Networks*, *19*(2), 143–155. https://doi.org/10.1016/S0378-8733(96)00297-3

Batagelj, V., Doreian, P., & Ferligoj, A. (1992). An optimizational approach to regular equivalence. *Social Networks*, *14*(1–2), 121–135. https://doi.org/10.1016/0378-8733(92)90016-Z

Batagelj, V., Ferligoj, A., & Doreian, P. (1992). Direct and indirect methods for structural equivalence. *Social Networks*, *14*(1–2), 63–90. https://doi.org/10.1016/0378-8733(92)90014-X

Batagelj, V., Ferligoj, A., & Doreian, P. (1998). Fitting Pre-specified Blockmodels. In C. Hayashi, K. Yajima, H.-H. Bock, N. Ohsumi, Y. Tanaka, & Y. Baba (Eds), *Data Science, Classification, and Related Methods* (pp. 199–206). Springer Japan. https://doi.org/10.1007/978-4-431-65950-1_20

Bijker, W. E. (1999). *Of bicycles, bakelites, and bulbs: Toward a theory of sociotechnical change* (3. Aufl). MIT Pr.

Bijker, W. E. . (1987). *The social construction of technological systems: New directions in the sociology and history of technology*. MIT press.

Borgatti, S. P., & Foster, P. C. (2003). The Network Paradigm in Organizational Research: A Review and Typology. *Journal of Management*, *29*(6), 991–1013. https://doi.org/10.1016/S0149-2063(03)00087-4

Borgatti, S. P., & Halgin, D. S. (2014). Analyzing Affiliation Networks. In J. Scott & P. Carrington, *The SAGE Handbook of Social Network Analysis* (pp. 417–433). SAGE Publications Ltd. https://doi.org/10.4135/9781446294413.n28

Brass, D. J., Galaskiewicz, J., Greve, H. R., & Tsai, W. (2004). Taking Stock of Networks and Organizations: A Multilevel Perspective. *The Academy of Management Journal*, *47*(6), 795–817. https://doi.org/10.2307/20159624

Browning, T. R. (2001). Applying the design structure matrix to system decomposition and integration problems: A review and new directions. *IEEE Transactions on Engineering Management*, *48*(3), 292–306. IEEE Transactions on Engineering Management. https://doi.org/10.1109/17.946528

Brusco, M., Doreian, P., & Steinley, D. (2021). Deterministic blockmodelling of signed and two-mode networks: A tutorial with software and psychological examples. *British Journal of Mathematical and Statistical Psychology*, *74*(1), 34–63. https://doi.org/10.1111/bmsp.12192

Bryson, A., Forth, J., & Stokes, L. (2017). Does employees' subjective well-being affect workplace performance? *Human Relations*, *70*(8), 1017–1037. https://doi.org/10.1177/0018726717693073

Burbidge, J. L. (1991). Production flow analysis for planning group technology. *Journal of Operations Management, Special Issue on Group Technology and Cellular Manufacturing*, *10*(1), 5–27. https://doi.org/10.1016/0272-6963(91)90033-T

Burns, T., & Stalker, G. M. (1961). *The management of innovation*. Tavistock.

Chandler, A. D. (1962). *Strategy and Structure: Chapters in the History of the American Industrial Enterprise*. MIT Press.

Cherns, A. (1976). The Principles of Sociotechnical Design. *Human Relations*, *29*(8), 783–792. https://doi.org/10.1177/001872677602900806

Collins, S. T., Bradley, J. A., & Yassine, A. A. (2010). Analyzing Product Development Task Networks to Examine Organizational Change. *IEEE Transactions on Engineering Management*, *57*(3), 513–525. https://doi.org/10.1109/TEM.2009.2033047

Cross, R. L., Parker, A., & Cross, R. (2004). *The Hidden Power of Social Networks: Understanding How Work Really Gets Done in Organizations*. Harvard Business Review Press.

Cugmas, M., Ferligoj, A., Škerlavaj, M., & Žiberna, A. (2021). Global structures and local network mechanisms of knowledge-flow networks. *PLoS ONE*, *16*(2), e0246660. https://doi.org/10.1371/journal.pone.0246660

Cugmas, M., Mali, F., & Žiberna, A. (2020). Scientific collaboration of researchers and organizations: A two-level blockmodeling approach. *Scientometrics*, *125*(3), 2471–2489. https://doi.org/10.1007/s11192-020-03708-x

Daita, S. T. S., Irani, S. A., & Kotamraju, S. (1999). Algorithms for production flow analysis. *International Journal of Production Research*, *37*(11), 2609–2638. https://doi.org/10.1080/002075499190680

de Sitter, L. U., Naber, J. L. G., & Verschuur, F. O. (1994). *Synergetisch Produceren. Human Resources Mobilisation in de produktie: Een inleiding in structuurbouw*. Van Gorcum.

Design and Structure Matrix (DSM). (2019, April 25). *The Dependency and Structure Modeling (DSM) Web*. https://dsmweb.org/design-structure-matrix-dsm/

Dessein, W., & Santos, T. (2006). Adaptive Organizations. *Journal of Political Economy*, *114*(5), 956–995. https://doi.org/10.1086/508031

DiMaggio, P. J., & Powell, W. W. (1983). The Iron Cage Revisited: Institutional Isomorphism and Collective Rationality in Organizational Fields. *American Sociological Review*, *48*(2), 147. https://doi.org/10.2307/2095101

Domain Mapping Matrix (DMM). (2019, April 25). *The Dependency and Structure Modeling (DSM) Web*. https://dsmweb.org/domain-mapping-matrix-dmm/

Doreian, P., Batagelj, V., & Ferligoj, A. (2004). Generalized blockmodeling of two-mode network data. *Social Networks*, *26*(1), 29–53. https://doi.org/10.1016/j.socnet.2004.01.002

Doreian, P., Batagelj, V., & Ferligoj, A. (2005). *Generalized Blockmodeling* (1st edn). Cambridge University Press. https://doi.org/10.1017/CBO9780511584176

DSM Conference. (2019, April 23). *Our Vision*. The Dependency and Structure Modeling (DSM) Web. https://dsmweb.org/our-vision-2/

Duxbury, S. W. (2024). Micro Effects on Macro Structure in Social Networks. *Sociological Methodology*, *54*(1), 1–26. https://doi.org/10.1177/00811750231209040

Eppinger, S. D., & Browning, T. R. (2012). *Design Structure Matrix Methods and Applications*. The MIT Press. https://doi.org/10.7551/mitpress/8896.001.0001

Ethiraj, S. K., & Levinthal, D. (2004). Bounded rationality and the search for organizational architecture: An evolutionary perspective on the design of organizations and their evolvability. *Administrative Science Quarterly*, *49*(3), 404–437. https://doi.org/https://doi.org/10.2307/413144

Everett, M. G., & Borgatti, S. P. (2020). Partitioning Multimode Networks. In P. Doreian, V. Batagelj, & A. Ferligoj, *Advances in network clustering and blockmodeling* (pp. 251–266).

Feld, S. L. (1981). The Focused Organization of Social Ties. *American Journal of Sociology*, *86*(5), 1015–1035. https://doi.org/10.1086/227352

Ferligoj, A., Doreian, P., & Batagelj, V. (2014). Positions and Roles. In J. Scott & P. Carrington, *The SAGE Handbook of Social Network Analysis* (pp. 417–433). SAGE Publications Ltd. https://doi.org/10.4135/9781446294413.n28

Freeman, L. C. (2014). The Development of Social Network Analysis – with an Emphasis on Recent Events. In J. Scott & P. Carrington (Eds), *The SAGE Handbook of Social Network Analysis* (pp. 26–39). SAGE Publications Ltd. https://doi.org/10.4135/9781446294413

Galbraith, J. R. (1974). Organization Design: An Information Processing View. *Interfaces*, *4*(3), 28–36.

Grant, R. M. (1996). Toward a Knowledge-Based Theory of the Firm. *Strategic Management Journal*, *17*, 109–122.

Greenwood, R., & Miller, D. (2010). Getting Back to the Heart of Organizational Theory. *Academy of Management Perspectives*.

Grossman, R., Nolan, K., Rosch, Z., Mazer, D., & Salas, E. (2022). The team cohesion-performance relationship: A meta-analysis exploring measurement approaches and the changing team landscape. *Organizational Psychology Review*, *12*(2), 181–238. https://doi.org/10.1177/20413866211041157

Hameri, A.-P. (2011). Production flow analysis—Cases from manufacturing and service industry. *International Journal of Production Economics*, *129*(2), 233–241. https://doi.org/10.1016/j.ijpe.2010.10.015

Hanneman, R. A., & Riddle, M. (2014). A Brief Introduction to Analyzing Social Network Data. In J. Scott & P. Carrington, *The SAGE Handbook of Social Network Analysis* (pp. 331–339). SAGE Publications Ltd. https://doi.org/10.4135/9781446294413.n23

Harter, J. K., Schmidt, F. L., Asplund, J. W., Killham, E. A., & Agrawal, S. (2010). Causal Impact of Employee Work Perceptions on the Bottom Line of Organizations. *Perspectives on Psychological Science*, *5*(4), 378–389. https://doi.org/10.1177/1745691610374589

Homans, G. C. (1950). *The human Group*. Harcourt, Brace & World.

Joseph, J., & Gaba, V. (2020). Organizational Structure, Information Processing, and Decision-Making: A Retrospective and Road Map for Research. *Academy of Management Annals*, *14*(1), 267–302. https://doi.org/10.5465/annals.2017.0103

Joseph, J., & Sengul, M. (2025). Organization Design: Current Insights and Future Research Directions. *Journal of Management*, *51*(1), 249–308. https://doi.org/10.1177/01492063241271242

Krackhardt, D., & Carley, K. (1998, June). A PCANS Model Structure in Organizations. *Proceedings of the 1998 International Symposium on Command and Control Research and Technology*. International Symposium on Command and Control Research and Technology.

Lawrence, P. R., & Lorsch, J. W. (1967). Differentiation and Integration in Complex Organizations. *Administrative Science Quarterly*, *12*(1), 1–47. https://doi.org/10.2307/2391211

Lehto, O., & Paniagua, P. (2025). Designing freedom: Allende, Pinochet and the twin experiments in cyber-socialism and neoliberalism. *Economy and Society*, *54*(2), 334–358. https://doi.org/10.1080/03085147.2025.2513800

March, J. G., & Simon, H. A. (1958). *Organizations*. Wiley.

Marin, A., & Wellman, B. (2014). Social Network Analysis: An Introduction. In J. Scott & P. J.Carrington (Eds), *The SAGE Handbook of Social Network Analysis* (pp. 11–25). SAGE Publications Ltd. https://doi.org/10.4135/9781446294413

Matjašič, M., Cugmas, M., & Žiberna, A. (2020). blockmodeling: An R package for generalized blockmodeling. *Metodoloski Zvezki*, *17*(2), 49–66. ProQuest Central (2776706933). https://doi.org/10.51936/uhir1119

Medina, E. (2011). *Cybernetic Revolutionaries: Technology and Politics in Allende's Chile*. MIT Press.

Miles, R. E., & Snow, C. C. (1978). *Organizational Strategy, Structure and Process*. McGraw-Hill.

Mintzberg, H. (1979). *The structuring of organizations: A synthesis of the research*. Prentice-Hall.

Morrison, R. M. (2021). *Data-Driven Organization Design: Delivering Perpetual Performance Gains Through the Organizational System* (2nd ed). Kogan Page, Limited.

Multiple-Domain Matrix (MDM). (2019, April 25). *The Dependency and Structure Modeling (DSM) Web*. https://dsmweb.org/multiple-domain-matrix-mdm/

Mumford, E. (with IGI Global). (2003). *Redesigning human systems*. IGI Global (701 E. Chocolate Avenue, Hershey, Pennsylvania, 17033, USA). https://doi.org/10.4018/978-1-59140-118-6

Nelson, R. E. (1986). The Use of Blockmodelling in the Study of Organization Structure: A Methodological Proposal. *Organization Studies*, *7*(1), 75–85. https://doi.org/10.1177/017084068600700105

Nelson, R. E., & Mathews, K. M. (1991). Cause Maps and Social Network Analysis in Organizational Diagnosis. *The Journal of Applied Behavioral Science*, *27*(3), 379–397. https://doi.org/10.1177/0021886391273009

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E*, *69*(2), 026113. https://doi.org/10.1103/PhysRevE.69.026113

Nordlund, C. (2016). A deviational approach to blockmodeling of valued networks. *Social Networks*, *44*, 160–178. https://doi.org/10.1016/j.socnet.2015.08.004

Nordlund, C. (2020). Direct blockmodeling of valued and binary networks: A dichotomization-free approach. *Social Networks*, *61*, 128–143. https://doi.org/10.1016/j.socnet.2019.10.004

Nordlund, C., & Žiberna, A. (2020). Blockmodeling of Valued Networks. In P. Doreian, V. Batagelj, & A. Ferligoj, *Advances in network clustering and blockmodeling* (pp. 151–188).

Parker, S. K., Van Den Broeck, A., & Holman, D. (2017). Work Design Influences: A Synthesis of Multilevel Factors That Affect the Design of Jobs. *Academy of Management Annals*, *11*(1), 267–308. https://doi.org/10.5465/annals.2014.0054

Pasmore, W. A. (1988). *Designing Effective Organizations: The Sociotechnical Systems Perspective*. Wiley.

Pasmore, W. A., Francis, C., Haldeman, J., & Shani, A. (1982). Sociotechnical Systems: A North American Reflection on Empirical Studies of the Seventies. *Human Relations*, *35*(12), 1179–1204. https://doi.org/10.1177/001872678203501207

Peixoto, T. P. (2020). Bayesian Stochastic Blockmodeling. In P. Doreian, V. Batagelj, & A. Ferligoj, *Advances in network clustering and blockmodeling* (pp. 289–332).

Powdthavee, N., & Frijters, P. (2017, January 30). Workers are happier with less hierarchy. *LSE Business Review*. https://blogs.lse.ac.uk/businessreview/2017/01/30/workers-are-happier-with-less-hierarchy/

Puranam, P. (2018). *The Microstructure of Organizations* (Vol. 1). Oxford University Press. https://doi.org/10.1093/oso/9780199672363.001.0001

Puranam, P. (2025). *Re-Humanize: How to build human-centric organizations in the age of algorithms*. Penguin Random House SEA.

Puranam, P., Raveendran, M., & Knudsen, T. (2012). Organization Design: The Epistemic Interdependence Perspective. *Academy of Management Review*, *37*(3), 419–440. https://doi.org/10.5465/amr.2010.0535

Raveendran, M., Silvestri, L., & Gulati, R. (2020). The Role of Interdependence in the Micro-Foundations of Organization Design: Task, Goal, and Knowledge Interdependence. *Academy of Management Annals*, *14*(2), 828–868. https://doi.org/10.5465/annals.2018.0015

Rivkin, J. W., & Siggelkow, N. (2003). Balancing Search and Stability: Interdependencies Among Elements of Organizational Design. *Management Science*, *49*(3), 290–311. https://doi.org/10.1287/mnsc.49.3.290.12740

Scott, J., & Carrington, P. (2014). *The SAGE Handbook of Social Network Analysis*. SAGE Publications Ltd. https://doi.org/10.4135/9781446294413

Sherman, J. D., & Keller, R. T. (2011). Suboptimal Assessment of Interunit Task Interdependence: Modes of Integration and Information Processing for Coordination Performance. *Organization Science*, *22*(1), 245–261. https://doi.org/10.1287/orsc.1090.0506

Simon, H. A. (1962). The Architecture of Complexity. *Proceedings of the American Philosophical Society*, *106*(6), 467–482.

Smith, A. (1776). *The wealth of nations: An inquiry into the nature and causes of the wealth of nations*. Harriman House Limited.

Stadtfeld, C., & Amati, V. (2021). Network mechanisms and network models. In *Research Handbook on Analytical Sociology* (pp. 432–452). Edward Elgar Publishing. https://www.elgaronline.com/edcollchap-oa/book/9781789906851/book-part-9781789906851-32.xml

Steward, D. V. (1981). The design structure system: A method for managing the design of complex systems. *IEEE Transactions on Engineering Management*, *EM-28*(3), 71–74. https://doi.org/10.1109/TEM.1981.6448589

Taylor, F. (1911). *The principles of scientific management*. Harper & Brothers.

Thompson, J. D. (1967). *Organizations in action: Social science bases of administrative theory.* McGraw-Hill.

Traag, V., Doreian, P., & Mrvar, A. (2020). Partitioning Signed Networks. In P. Doreian, V. Batagelj, & A. Ferligoj, *Advances in network clustering and blockmodeling* (pp. 225–250).

Tushman, M. L., & Nadler, D. A. (1978). Information Processing as an Integrating Concept in Organizational Design. *The Academy of Management Review*, *3*(3), 613–624. https://doi.org/10.2307/257550

Van Hootegem, G. (2019). De moderne sociotechniek: De weg naar Total Workplace Innovation. In L. van Oss & B. Kessener, *Meer dan de som der delen: Systeemdenkers over organiseren en veranderen*. Vakmedianet Management B.V.

Worren, N., & Pope, S. (2024). Connected but Conflicted: Separating Incompatible Roles in Organizations. *Academy of Management Review*, *49*(1), 6–31. https://doi.org/10.5465/amr.2021.0054

Zhou, Y. M. (2013). Designing for Complexity: Using Divisions and Hierarchy to Manage Complex Tasks. *Organization Science*, *24*(2), 339–355. https://doi.org/10.1287/orsc.1120.0744

Žiberna, A. (2007). Generalized blockmodeling of valued networks. *Social Networks*, *29*(1), 105–126. https://doi.org/10.1016/j.socnet.2006.04.002

Žiberna, A. (2008). Direct and Indirect Approaches to Blockmodeling of Valued Networks in Terms of Regular Equivalence. *The Journal of Mathematical Sociology*, *32*(1), 57–84. https://doi.org/10.1080/00222500701790207

Žiberna, A. (2009). Evaluation of direct and indirect blockmodeling of regular equivalence in valued networks by simulations. *Advances in Methodology and Statistics*, *6*(2). https://doi.org/10.51936/kshx3707

Žiberna, A. (2014). Blockmodeling of multilevel networks. *Social Networks*, *39*, 46–61. https://doi.org/10.1016/j.socnet.2014.04.002

Žiberna, A. (2020). Blockmodeling Linked Networks. In P. Doreian, V. Batagelj, & A. Ferligoj, *Advances in network clustering and blockmodeling* (pp. 267–287).